

Architecture as Geometric Prior: A Three-Primitive Framework for Manifold-Aware Neural Network Design

Elias Natti

April 15, 2026

Abstract

We show that all known neural network architectures—convolutional, attentional, graph, spherical, and hyperbolic—are specific constraint configurations of three known reversible primitives: permutation, pointwise invertible transform, and invertible mixer. Each primitive operates in an *intrinsic mode* (within a fixed space) and an *extrinsic mode* (changing the space itself). Intrinsic completeness and minimality follow from classical results (LU decomposition, butterfly factorization). The extrinsic modes parameterize smooth, invertible paths on Grassmannian manifolds, reframing dimensional changes not as lossy projections but as subspace rotations. Together, these yield a unified characterization: a network’s computation is a path on the Grassmannian from the data’s initial subspace to the task’s meaning space, and an architecture’s constraints determine which paths are accessible. We demonstrate that this characterization recovers every major architecture family as a point in a continuous, parameterizable constraint space. Dissolving the traditional separation between position and features reveals that convolution and attention are different constraints on the same mixer, not different operations. Experimentally, we show that the $\exp(A)$ parameterization provides intrinsic nonlinearity that subsumes explicit activation functions, with phase decomposition into symmetric (scaling) and skew-symmetric (rotation) components achieving three orders of magnitude improvement over GELU on periodic tasks. We further reinterpret latent space as geometric mismatch on the Grassmannian and decompose it into task-productive and task-irrelevant components, unifying discriminative and generative models as opposite traversal directions on the same manifold. The central implication is that architecture development itself—historically a process of human intuition and empirical search—can be reformulated as optimization over a well-defined geometric space. We compare this primitives-based approach to the symmetry-group framework of geometric deep learning, arguing that symmetry groups characterize what an architecture preserves while primitives characterize what an architecture assumes and computes.

1 Introduction

The choice of neural network architecture—convolution, attention, message passing, recurrence—is among the most consequential design decisions in machine learning. Yet the theoretical frameworks for understanding this choice remain fragmented. Practitioners select architectures based on empirical benchmarks and domain conventions, with limited formal guidance on *why* one architecture suits a problem better than another.

The geometric deep learning program [Bronstein et al., 2021] provides the most comprehensive unifying framework to date, organizing architectures by their symmetry groups. CNNs are equivariant to translation, GNNs to permutation, spherical CNNs to rotation, and so on. This is powerful

for architecture *derivation*—given a known symmetry, one can derive the most general equivariant architecture. However, symmetry groups characterize what the architecture *preserves*, leaving open the question of what the architecture *does* computationally and what geometric assumptions it encodes.

We propose a complementary framework that addresses this gap. Our central claim:

An architecture does not learn a manifold. An architecture *is* a geometric prior—a specific set of constraints on a universal set of computational primitives that determines which Grassmannian paths from data space to meaning space are accessible. Training selects the specific path within the constrained family. The designer’s task is to choose constraints that make the shortest path available—which means matching the constraints to the data’s true geometric structure.

The framework rests on three reversible primitives—permutation, pointwise transform, and invertible mixer—whose intrinsic completeness and minimality are known results from linear algebra (LU decomposition, butterfly factorization). We do not claim to introduce new primitives. Our contribution is to show that these primitives, extended with extrinsic modes that operate on the Grassmannian, provide a universal characterization of all known architectures as points in a single continuous constraint space. The constraints imposed on these primitives fully determine the architecture’s geometric assumptions: which pairs of elements interact (topology), how interactions are parameterized (metric and curvature), and what symmetries are enforced (equivariance). Different architectures are different constraint patterns on the same primitives—and this means architecture development itself can be parameterized and learned.

This should be distinguished from neural architecture search methods such as DARTS [Liu et al., 2019], which relax a discrete search over candidate *operations*—convolution, max pooling, skip connection, zero—into a continuous optimization. DARTS and its descendants search over a hand-designed menu of composite building blocks; the search space is defined by the operations the designer chooses to include. The present framework operates one level deeper: the “operations” that DARTS treats as atomic are themselves decomposable into constraint configurations on the three primitives. The search space is not a menu but a continuous manifold derived from first principles, and architecture search becomes navigation on this manifold rather than selection from a predefined list.

Contributions.

1. We show that three known reversible primitives—permutation, pointwise transform, and invertible mixer—with intrinsic and extrinsic modes, universally characterize all major neural network architecture families as specific constraint configurations (Sections 2–4).
2. We demonstrate that the extrinsic modes parameterize invertible paths on Grassmannian manifolds, providing a geometric interpretation of dimensional changes as subspace rotations rather than lossy projections (Section 2).
3. We show that dissolving the position/feature distinction reveals that convolution and attention are constraints on the same mixer, not different operations (Section 4).
4. We characterize the resulting constraint space as continuous and searchable, implying that architecture development—historically driven by human intuition—can be reformulated as optimization over a well-defined geometric space (Section 3).

5. We compare the primitives framework to the symmetry-group framework, clarifying their complementary roles (Section 5).
6. We provide experimental validation that the $\exp(A)$ parameterization supplies intrinsic non-linearity matching explicit activation functions, with phase decomposition into symmetric and skew-symmetric components revealing that scaling and rotational modes serve distinct geometric roles—achieving three orders of magnitude improvement over GELU on periodic tasks while matching it on classification. We identify and resolve the eigenvalue instability of $\exp(A)$ via a spectral clamp on the Lie algebra (Section 6).
7. We reinterpret latent space as geometric mismatch on the Grassmannian, decompose off-diagonal content into parallel (task-productive) and perpendicular (task-irrelevant) components, and show that this decomposition unifies discriminative and generative models as opposite traversal directions on the same manifold—with implications for transfer learning, architecture–task matching, and the role of data dimensionality in the $k \times k$ operating space (Sections 7 and 8).

2 The Three Primitives

2.1 The Unified State Vector

We represent each data element as a single state vector $s_i \in \mathbb{R}^d$. Some dimensions may encode spatial position, temporal index, or other structured quantities, but this is not enforced at the primitive level. The state vector is the complete description of the element.

This dissolves the traditional (v_i, p_i) representation where features v_i and position p_i are separate objects requiring separate operations. We return to the implications of this dissolution in Section 4.

2.2 Primitive 1: Permutation

Definition 1 (Permutation Primitive). *A bijection $\sigma : \{1, \dots, N\} \rightarrow \{1, \dots, N\}$ applied to the index set of state vectors:*

$$P_\sigma : s_i \mapsto s_{\sigma(i)} \quad (1)$$

The permutation reorders state vectors without modifying them. It is always invertible: $P_\sigma^{-1} = P_{\sigma^{-1}}$.

Intrinsic mode (routing within fixed state set). The permutation determines *which pairs of states interact* at each layer. The pattern of permutations across layers defines the architecture’s assumed **topology**. Different patterns create qualitatively different connectivity: stride permutations (butterfly pattern) achieve $O(\log N)$ diameter; shift permutations achieve local, translation-equivariant connectivity; data-dependent soft permutations (as in attention) allow the topology to be inferred from content.

Extrinsic mode (cross-block routing). The extrinsic permutation changes *which blocks can communicate*—it defines the macro-topology of the network. In the single-layer geometric view, where the full network is one element of $\text{GL}(D)$ factored into $k \times k$ blocks along the diagonal, intrinsic permutation routes rows within a block; extrinsic permutation routes rows *across* blocks, breaking the strictly sequential block-diagonal structure. The residual connection [He et al., 2016] is the simplest case: an identity permutation that copies each block’s input past the block, allowing

block i to communicate directly with block $i + 2$. Dense connections [Huang et al., 2017b] make the cross-block routing fully connected: every block receives all previous blocks’ outputs. Encoder-decoder bridges [Ronneberger et al., 2015] create symmetric cross-block topology, routing block i in the encoder to block $L - i$ in the decoder. The intrinsic mode rewires a graph with fixed nodes; the extrinsic mode rewires the graph *of graphs*—the connectivity between layers themselves.

2.3 Primitive 2: Pointwise Invertible Transform

Definition 2 (Pointwise Transform Primitive). *At each index i , apply an invertible linear map $G_i \in \text{GL}(d)$:*

$$\mathcal{G} : s_i \mapsto G_i \cdot s_i \quad (2)$$

Invertible when $\det(G_i) \neq 0$ for all i . Inverse: $G_i^{-1} \cdot s_i$. Special cases include diagonal scaling ($G_i = \text{diag}(\alpha_1, \dots, \alpha_d)$), local rotation, and gauge transformations in the fiber bundle sense of Weiler et al. [2021].

Intrinsic mode (frame change within fixed dimensionality). This encodes the architecture’s assumed **local frame**. If $G_i = G$ for all i (shared), the architecture assumes homogeneity. If G_i varies, the architecture allows position-dependent representation.

Extrinsic mode (dimensionality change). The extrinsic pointwise transform changes the *dimensionality* of each state vector by embedding into a higher-dimensional space ($\mathbb{R}^d \rightarrow \mathbb{R}^D$ with $D > d$) or projecting to a lower-dimensional one. This is where a point becomes a vector in a larger space—the space *around* the point changes, not just the point itself. Multi-head attention’s W_Q, W_K, W_V projections from d into $N \times d_k$ total dimensions are extrinsic pointwise transforms, as is the output projection W_O that contracts back. In the continuous limit, dimensionality change is parameterized by smooth motion on the Grassmannian $\text{Gr}(k, D)$ —the manifold of k -dimensional subspaces of $\mathbb{R}^D$ —allowing the representation to grow smoothly into new orthogonal dimensions as the data’s intrinsic curvature demands, rather than jumping between fixed layer widths.

2.4 Primitive 3: Invertible Mixer

Definition 3 (Mixer Primitive). *For a designated group of m states $(s_{i_1}, \dots, s_{i_m})$, apply an invertible $md \times md$ block:*

$$\mathcal{B} : \begin{pmatrix} s_{i_1} \\ \vdots \\ s_{i_m} \end{pmatrix} \mapsto M \begin{pmatrix} s_{i_1} \\ \vdots \\ s_{i_m} \end{pmatrix}, \quad M \in \text{GL}(md) \quad (3)$$

where M may depend on the states themselves. The smallest nontrivial case is $m = 2$ (pairwise mixing).

Invertible when $\det(M) \neq 0$. In practice, invertibility is guaranteed by parameterizing $M = \exp(A)$ where $A \in \mathfrak{gl}(md)$ is unconstrained—the matrix exponential maps from the Lie algebra to $\text{GL}(md)$, ensuring $\det(M) > 0$ for any A , with inverse $M^{-1} = \exp(-A)$. When A is data-dependent (generated by a network from the states), the exponential parameterization also introduces nonlinearity through its series expansion, rendering separate activation functions unnecessary. This is the core computational primitive—it couples m state vectors, enabling each output to depend on all m inputs simultaneously, in a balanced way that preserves invertibility.

The mixer encodes the architecture’s assumed **interaction law**. The functional form of M —what it depends on and how—determines:

- **What constitutes proximity:** if M_{ij} depends only on position-dimensions of the states, the architecture assumes spatial geometry. If it depends on all dimensions equally, the architecture assumes content-based geometry.
- **What symmetries hold:** if M_{ij} depends only on $s_i - s_j$ (restricted to position dimensions), translation invariance is assumed.
- **What curvature exists:** determined by the holonomy of composed mixers around closed loops (Remark 1 below).

Intrinsic mode (coupling within fixed interaction space). The intrinsic mixer couples m states within a feature space of fixed dimension d , with fixed interaction order m : $\exp(A)$ for $A \in \mathfrak{gl}(md)$. The convolution kernel ($m = 2$, pairwise), attention-weighted sum ($m = N_{\text{tokens}}$, full-sequence), and message-passing update ($m = |\mathcal{N}(i)|$, neighborhood) are all intrinsic mixers at different interaction orders: they combine information from participants without changing the representation’s dimensionality.

Extrinsic mode (interaction order). The extrinsic mixer changes m —how many states participate in a single $\exp(A)$. Going from $A \in \mathfrak{gl}(md)$ to $A \in \mathfrak{gl}(m'd)$ with $m' > m$ enlarges the Lie algebra. The matrix exponential of the full $m'd \times m'd$ matrix produces cross-terms between all m' states that no finite composition of m -body $\exp(A)$ s can reproduce—the series expansion $I + A + A^2/2! + \dots$ generates products of off-diagonal blocks across all participants simultaneously.

Most current architectures fix $m = 2$ (pairwise) and use depth to approximate higher-order interactions. Attention is the exception: the softmax-weighted sum is a genuinely N -body operation because the normalization $\exp(q \cdot k_j) / \sum_n \exp(q \cdot k_n)$ makes each output depend on all inputs simultaneously, a global dependency that pairwise steps cannot factor. Multi-head attention’s power is often attributed to multiple independent channels, but the channel structure is pointwise extrinsic (the W_Q, W_K, W_V projections create the dimensional room); the mixer’s extrinsic contribution is the interaction order within each head’s softmax. In the continuous limit, m becomes a smooth parameter governing how many states the data’s geometry requires to see simultaneously at each layer.

The three primitives thus maintain a clean separation at both levels: pointwise decides *what points exist* (dimensions), permutation decides *which points can talk* (connectivity), and the mixer decides *how points talk* (interaction law and order).

Remark 1 (Curvature from Primitive Composition). *Curvature does not require a separate primitive—it emerges from the composition of mixers around closed loops. For a closed loop of states s_1, s_2, s_3 , the discrete holonomy is $\text{Hol}(s_1, s_2, s_3) = M_{31} \circ M_{23} \circ M_{12}$. On a flat manifold $\text{Hol} = I$ for every triangle; on a curved manifold $\text{Hol} \neq I$, and the deviation from identity measures the enclosed curvature. The flatness condition $\text{Hol} = I$ is a constraint on the mixer parameters: an architecture with translation-invariant mixers (CNN) automatically satisfies it—flatness is imposed, not discovered—while independently parameterized mixers (GNN) can violate it, making curvature representable but not imposed. This is the discrete analogue of lattice gauge theory, where the gauge field on a lattice is described by group elements on edges and curvature is measured by the product around elementary plaquettes.*

2.5 Completeness

Theorem 1 (Intrinsic Completeness). *Any matrix $A \in \text{GL}(Nd)$ can be expressed as a finite product of permutation matrices, block-diagonal matrices of pointwise transforms, and block-diagonal matrices of $m = 2$ (pairwise) mixers.*

This is a restatement of standard results in the language of our primitives. LU decomposition with partial pivoting gives $A = PLU$ where P is a permutation, L is lower triangular, and U is upper triangular. Each off-diagonal entry in L and U is an elementary row operation—a shear—which is a pairwise mixer with one off-diagonal block set to zero (a degenerate mixer). The diagonal entries are pointwise scaling. Since any $A \in \text{GL}(Nd)$ admits LU decomposition, and each factor maps to one of our three primitives, completeness follows. The general decomposition requires $O(Nd)$ factors.

Remark 2 (Depth efficiency for structured matrices). *For matrices with known structure—circulant (convolutions), Hadamard (FFTs), and their products—[Dao et al. \[2020\]](#) show that $O(\log N)$ butterfly layers suffice, where each butterfly layer is a permutation composed with block-diagonal 2×2 invertible blocks. The $O(\log N)$ bound applies to the Kaleidoscope class; we do not claim it for arbitrary $\text{GL}(Nd)$.*

2.6 Minimality

Proposition 1 (Minimality). *Removing any one primitive type from the set makes it incomplete. Each case has a different character—algebraic, topological, and geometric respectively—reflecting the distinct role of each primitive.*

Case 1: Without mixers (algebraic). Permutations and pointwise transforms generate only monomial matrices—products of a permutation and a diagonal matrix. Monomial matrices form a proper subgroup of $\text{GL}(Nd)$ that cannot produce off-diagonal coupling between state vectors. Any matrix with nonzero off-diagonal entries in the same block row is unreachable.

Case 2: Without permutations (topological). Pointwise transforms and mixers can only couple states that are adjacent in the initial ordering. The interaction graph’s connectivity is fixed by the initial grouping structure. Reaching arbitrary pairs requires $O(N)$ depth in the worst case, and some interaction topologies—those requiring non-local routing—cannot be achieved at any depth without reordering.

Case 3: Without pointwise transforms (geometric). Intrinsically, a degenerate mixer (one off-diagonal block set to zero) can simulate pointwise scaling, so the algebraic case is subtle—minimality depends on requiring genuine coupling in the mixer definition. Extrinsically, the argument is clean: only the pointwise transform’s extrinsic mode changes the dimensionality d of each state vector. Neither permutations (which change count N) nor mixers (which change interaction order m) can alter d . Removing pointwise transforms makes dimensional embedding and projection impossible.

Remark 3. *The three minimality cases have fundamentally different characters. Case 1 is algebraic (monomial $\subsetneq \text{GL}$). Case 2 is topological (bounded connectivity). Case 3 is geometric (inability to change the space’s dimension). This reflects the distinct roles of the three primitives: mixers provide algebraic coupling, permutations provide topological routing, and pointwise transforms provide geometric embedding.*

3 The 3×2 Grid: Intrinsic and Extrinsic Modes

Each primitive operates in two modes—intrinsic (within a fixed space) and extrinsic (changing the space itself)—yielding the 3×2 grid in Table 1.

Table 1: The 3×2 grid. Each primitive has an intrinsic mode that operates within a fixed-dimensional, fixed-count state space, and an extrinsic mode that changes the space itself.

Primitive	Intrinsic (within space)	Extrinsic (changes space)
Permutation	Reorder/route within block	Route across blocks (macro-topology)
Pointwise Transform	Point $\rightarrow$ point (same d)	Embed/project ($d \rightarrow D$)
Mixer	$\exp(A)$, $A \in \mathfrak{gl}(md)$, fixed m	Change m (interaction order)

The intrinsic modes are the “standard” operations: routing (which states interact), frame changes (how each state is represented), and coupling (how pairs exchange information). The extrinsic modes capture the architecture’s macro-structure: embedding dimension changes (what points exist), cross-block connectivity (which points can talk), and interaction order (how many points participate in a single $\exp(A)$).

Data-dependent parameters. Any primitive can have either fixed parameters (set at design time or learned) or data-dependent parameters (computed from the input at inference time). The data-dependence axis is orthogonal to the 3×2 grid; it describes *how parameters are set*, not *which primitive* is being applied. Table 2 illustrates the distinction across all six cells.

Table 2: Data dependence is orthogonal to the 3×2 grid. Each cell can operate with fixed parameters (set at design time or learned once) or with data-dependent parameters (computed from the input at inference time).

Primitive	Mode	Fixed example	Data-dependent example
Permutation	Intrinsic	Stride/shift pattern	Attention routing
Permutation	Extrinsic	Residual skip (fixed topology)	Learned skip gating
Pointwise	Intrinsic	Learned linear layer	Layer/batch normalization
Pointwise	Extrinsic	W_Q, W_K, W_V projections	Gating (GLU, MoE routing)
Mixer	Intrinsic	Convolution kernel	Attention-weighted sum
Mixer	Extrinsic	Fixed interaction order (pairwise)	Adaptive k -body grouping

Attention is the canonical data-dependent permutation: the routing is computed from the data. Attention weights are a data-dependent mixer: the coupling strength depends on content. Normalization layers—batch normalization, layer normalization, RMSNorm—are data-dependent pointwise transforms: they compute statistics (mean, variance) from the activations and use these to rescale each dimension.

Normalization as Grassmannian stabilization. In the Grassmannian interpretation, normalization layers prevent the representation from drifting into degenerate regions of $\text{Gr}(k, D)$ where some dimensions collapse to near-zero activation. Without normalization, the occupied subspace can become ill-conditioned—effectively losing dimensions as certain directions die out. Normalization rescales all active dimensions to comparable magnitude, constraining the Grassmannian path

to the well-conditioned interior where all k active dimensions remain geometrically meaningful. This explains why normalization enables higher learning rates (larger Grassmannian steps are safe because normalization prevents landing in degenerate regions) and why it reduces sensitivity to initialization (the Grassmannian path is pulled back toward a canonical position within its current subspace after each step, regardless of where it started).

3.1 Multi-Head Attention as a 3×2 Decomposition

Multi-head attention—which appears to require a “parallel composition” primitive for its N independent heads—decomposes cleanly into the 3×2 grid:

1. **Pointwise transform** (extrinsic, expand): The W_Q, W_K, W_V projections map each token’s d -dimensional state into $N \times d_k$ total dimensions, creating N independent subspaces. This is an extrinsic pointwise transform—the representation temporarily expands into a higher-dimensional ambient space.
2. **Permutation** (intrinsic, data-dependent): Within each head, the attention weights $\text{softmax}(Q_h K_h^T / \sqrt{d_k})$ define a soft, data-dependent permutation over tokens. This is intrinsic: no dimensionality or count changes, only routing.
3. **Mixer** (intrinsic per head, extrinsic via softmax): Each head computes a weighted sum in its d_k -dimensional subspace—an intrinsic mixer. The N independent heads are a consequence of the pointwise extrinsic expansion creating room for N copies, not a separate extrinsic mode. The mixer’s extrinsic contribution is the softmax normalization, which makes each output depend on all N_{tokens} inputs simultaneously—a genuinely k -body interaction that cannot be decomposed into pairwise steps.
4. **Pointwise transform** (extrinsic, contract): The output projection W_O maps the concatenated head outputs from $N \times d_v$ back to d dimensions. The representation contracts back.

The “parallel processing” of multiple heads is not a fourth primitive but the natural result of the pointwise extrinsic mode: expand the dimensionality to create N independent subspaces, apply intrinsic mixers within each, and compress back. The head count is a pointwise parameter, not a mixer parameter. The mixer’s own extrinsic contribution—the k -body interaction order of the softmax—is what gives attention its expressive power beyond what N independent pairwise couplings would provide.

3.2 A Full Transformer Block

A complete transformer block (pre-norm variant) decomposes as:

1. **Pointwise** (data-dependent): Layer normalization—stabilizes the Grassmannian position by rescaling all active dimensions to comparable magnitude
2. **Pointwise** (extrinsic): W_Q, W_K, W_V projections ($d \rightarrow Nd_k$)
3. **Permutation** (data-dependent): Attention weights per head
4. **Mixer** (extrinsic, N channels): Weighted sum per head
5. **Pointwise** (extrinsic): W_O projection ($Nd_v \rightarrow d$)

6. **Permutation** (extrinsic): Residual connection—routes input past the attention block
7. **Pointwise** (data-dependent): Layer normalization—re-stabilizes after residual addition
8. **Pointwise** (extrinsic): FFN up-projection ($d \rightarrow 4d$)
9. **Pointwise**: Nonlinear activation (GELU/SiLU)
10. **Pointwise** (extrinsic): FFN down-projection ($4d \rightarrow d$)
11. **Permutation** (extrinsic): Residual connection—routes input past the FFN block

Every component maps to a specific primitive in a specific mode. The residual connections are extrinsic permutations: they route each block’s input past the block, creating the cross-block connectivity that defines the transformer’s macro-topology. The FFN’s up/down projections are extrinsic pointwise transforms that temporarily expand the representation, mirroring the multi-head pattern at the individual-token level. The two normalization layers serve as Grassmannian guardrails: after each major operation (attention, residual addition), they project the representation back to a well-conditioned region of the occupied subspace, ensuring that subsequent layers receive geometrically stable input even as the subspace itself rotates across layers.

3.3 Implications for Automated Architecture Design

The 3×2 grid reduces architecture design to choosing six continuous parameters per layer:

- **Permutation intrinsic**: connectivity pattern (local vs. global, stride, data-dependent)
- **Permutation extrinsic**: cross-block topology (residual, dense, encoder-decoder bridges)
- **Pointwise intrinsic**: frame sharing (homogeneous vs. position-dependent)
- **Pointwise extrinsic**: embedding dimension
- **Mixer intrinsic**: interaction law (displacement-only, content-based, hybrid)
- **Mixer extrinsic**: interaction order (k -body coupling)

Geometric analysis of the dataset determines these parameters: the data’s intrinsic dimensionality sets the embedding dimension, its curvature structure determines the number of independent metrics needed (channel count), its topology determines the within-layer connectivity pattern, the depth of geometric transformation required determines the cross-block topology, and its symmetries determine the degree of weight sharing. Current architectures discretize these parameters; the framework treats them as continuous.

4 Architectures as Constraint Patterns

The unified state vector dissolves the position/feature distinction, revealing that convolution and attention are *different constraints on the same mixer*, not different mathematical operations. Convolution constrains the mixer to depend only on position-dimensions with translation invariance: $M_{ij} = M(s_i^{(\text{pos})} - s_j^{(\text{pos})})$. Attention places no constraint on which dimensions the mixer depends on: $M_{ij} = f(s_i, s_j)$ for arbitrary learned f . The difference is the *constraint*, not the primitive. This is why architectures that “combine” convolution and attention (e.g., convolutional attention, local attention windows) are natural—they apply different constraint levels to the same mixer at different scales or for different dimension subsets.

4.1 The Constraint Hierarchy

Architectures can be ordered by how tightly they constrain the primitive set. Table 3 shows this ordering from least to most constrained.

Table 3: Architecture families ordered by constraint strength. Moving down: stronger geometric prior, fewer parameters, more data-efficient, more restricted manifold class.

Architecture	Connectivity	Sharing	Mixer form	Geometric prior
MLP	All-to-all	Per-layer	State-independent	None
Attention	All-to-all	Shared W_Q, W_K, W_V	Data-dependent	Permutation-invariant
GNN	Graph edges	Shared message fn	State + edge	Fixed topology
Spherical CNN	Geodesic ball	$SO(3)$ -equivariant	Zonal	Sphere ($\kappa > 0$)
CNN	Local grid	Translation-invariant	Displacement-only	Flat Euclidean

4.2 Detailed Constraint Analysis

CNN: Flat Euclidean Prior. The permutation follows the butterfly pattern of the 2D FFT. The pointwise transform is shared across all positions ($G_i = G$), enforcing homogeneity. The mixer depends only on position-dimension differences: $M_{ij} = M(s_i^{(\text{pos})} - s_j^{(\text{pos})})$, enforcing translation invariance. Position dimensions pass through unchanged. These constraints collectively assume the manifold is $\mathbb{R}^2$ (or $\mathbb{T}^2$ for circular convolution) with flat Euclidean metric.

Attention: Minimal Geometric Prior. No connectivity constraint (all pairs interact). Shared projections W_Q, W_K, W_V across all positions (weak homogeneity). The mixer is data-dependent: $M_{ij} \propto \exp(s_i^T W_Q^T W_K s_j / \sqrt{d})$, with no privileged dimensions. This assumes no geometry—all structure must be learned. Positional encoding partially restores the position/feature distinction by injecting position information into the state, compensating for the architecture’s refusal to treat any dimensions as special.

GNN: Topological Prior. Connectivity constrained to graph edges. The mixer depends on node states and edge features: $M_{ij} = f(s_i, s_j, e_{ij})$. Topology is assumed known; metric and curvature are learned or provided through edge features. Each message-passing step is one layer of mixing along graph edges ($m = |\mathcal{N}(i)|$).

Spherical CNN: Curved Homogeneous Prior. Permutation follows the spherical harmonic transform structure. The pointwise transform is $SO(3)$ -equivariant. The mixer depends only on geodesic distance (zonal kernels). These constraints assume the manifold is S^2 with constant positive curvature and $SO(3)$ symmetry.

Hyperbolic Networks: Negative Curvature Prior. Permutation follows hyperbolic tiling structure. The pointwise transform includes the conformal factor $(1 - \|x\|^2)^{-2}$. The mixer uses Möbius operations for parallel transport. These constraints assume the manifold is $\mathbb{H}^n$ with constant negative curvature—natural for hierarchical data due to exponential volume growth.

Normalization: Grassmannian Path Regularization. Normalization layers (batch norm, layer norm, RMSNorm) are not architecture-specific—they appear across all families. In the primitives framework, they are data-dependent pointwise transforms that constrain the Grassmannian path rather than directing it. Without normalization, the representation can drift into degenerate subspaces where some dimensions collapse and others explode, effectively reducing the manifold’s accessible dimensionality. Normalization rescales all active dimensions to comparable magnitude after each computational step, acting as a projection back to the well-conditioned interior of the current Grassmannian position. This explains the empirical observation that normalization enables faster convergence: it prevents the Grassmannian path from wasting steps escaping degenerate regions. It also explains internal covariate shift geometrically—as earlier layers rotate their output subspace during training, later layers find their input at an unexpected Grassmannian position; normalization re-centers the representation within the current subspace, decoupling each layer’s subspace rotation from the stability of subsequent layers.

4.3 The Design Principle

The architectural design problem reduces to: *choose the tightest constraint pattern that is consistent with the data’s true geometric structure.*

Too few constraints (attention for grid data) wastes data—the architecture must learn geometric structure that could have been provided for free. Too many constraints (CNN for graph data) misrepresents the geometry—the architecture cannot express the true manifold. The optimal architecture matches its constraint set to the data manifold.

5 Comparison: Symmetry Groups vs. Computational Primitives

The geometric deep learning framework [Bronstein et al., 2021] and our primitives framework address different questions about the same architectures.

Different organizing principles. Symmetry groups characterize what the architecture *preserves* (equivariance). Primitives characterize what the architecture *does* (computation) and *assumes* (geometry). The symmetry approach is top-down: specify the desired invariance, derive the architecture. The primitives approach is bottom-up: specify the computational building blocks and their constraints, analyze the resulting geometric assumptions.

Different granularity. Two architectures can be equivariant to the same symmetry group but have very different computational decompositions—different depth, different connectivity patterns, different fan-out/fan-in structure. The symmetry framework treats them as equivalent (same equivariance class). The primitives framework distinguishes them (different primitive compositions). A dense $SO(3)$ -equivariant layer and its spherical harmonic butterfly factorization are identical from the symmetry perspective but structurally different in primitives: the butterfly version has multi-scale locality that the dense version lacks.

Different strengths. The symmetry approach excels at architecture *design*: given a known symmetry, it provides a constructive procedure for building equivariant architectures. The primitives approach excels at architecture *analysis*: given an architecture, it reveals the geometric assumptions encoded in its constraint pattern. For architecture *search*, the primitives framework offers a

continuous parameterization (constraint levels on each primitive) that the discrete symmetry-group framework does not.

Complementary, not competing. A complete analysis of an architecture benefits from both perspectives. Symmetry tells you *what class of functions* the architecture lives in. Primitives tell you *where in that class* the architecture sits, *which functions are easy to represent*, and *what the computational cost is*. The symmetry framework specifies which transformations the architecture must respect; the primitives framework specifies how the architecture navigates the Grassmannian from data space to meaning space, and what constraints shape that navigation.

Automatic symmetry discovery via Lie algebra parameterization. A complementary line of work parameterizes neural network layers directly in the Lie algebra, learning which symmetry group the data respects rather than assuming one. Dehmamy et al. [2021] introduce L-conv, which learns the generators of the symmetry group from data by parameterizing convolutional filters through the matrix exponential of learned Lie algebra elements. This shares a core mechanism with our mixer primitive, which is also parameterized as $\exp(A)$ with $A \in \mathfrak{gl}(md)$. The difference is scope: L-conv uses the Lie algebra parameterization to discover symmetries *within* a fixed convolutional scaffolding, while we use the same parameterization as the unconstrained baseline from which architectures are obtained by *adding* constraints. In the primitives framework, L-conv corresponds to the intrinsic mode of the mixer (fixed m , learn the generator) operating within a convolution-constrained permutation pattern. The two frameworks are complementary: L-conv provides a principled procedure for discovering the generator structure our mixer should learn, and our framework provides the broader computational geometry in which such discoveries fit.

Relation to existing Grassmannian approaches. Grassmannian geometry already appears in deep learning, but in a different role than the one we propose. Networks *on* Grassmannians [Huang et al., 2017] process data that intrinsically lives on $\text{Gr}(k, n)$ —video clips as subspaces, image sets as span representations—by designing layers (projection, pooling, orthonormalization) that respect Grassmannian structure. The layer types they introduce overlap structurally with the three primitives, though constrained to orthonormality and treated as specialized components for Grassmannian data rather than as universal building blocks. That independent architecture design for Grassmannian geometry arrives at a similar decomposition supports the claim that the three primitive types are fundamental. More recently, Chong [2025] replace attention with Grassmannian mixing, interpreting token pairs as 2-dimensional subspaces on $\text{Gr}(2, r)$ and embedding them via Plücker coordinates. Their key insight—that “semantic reasoning can emerge from controlled deformations of low-rank subspaces across layers”—is consonant with our framework. The distinction is scope: these works use Grassmannian geometry *within* a specific architecture (as data representation or as a mixing mechanism), whereas we use it *to interpret all architectures*. In our framework, the Grassmannian is not an architectural component but the space in which every architecture’s computation takes place—the extrinsic modes of the three primitives generate paths on it, and the architecture’s constraints determine which paths are accessible. The Grassmann flow mixer of Chong [2025] is, in our language, a specific constraint pattern on the mixer’s extrinsic mode.

6 Experimental Illustrations

The preceding sections develop the framework theoretically. We now present two experiments that visualize the central claims: that convolution and attention are constraint configurations of a

single mixer, and that representational dimensionality change corresponds to smooth motion on the Grassmannian.

6.1 The Unified Mixer: Convolution and Attention as Constraint Configurations

We construct a 16-position input sequence with two planted motifs (motif A at positions $\{0, 1, 8, 9\}$, motif B at positions $\{4, 5, 12, 13\}$) and weak noise elsewhere. This creates a specific disagreement: positions sharing a motif are content-similar but spatially distant, forcing different constraint settings to produce visibly different mixing behavior.

A single mixer instance computes $\text{output}_i = \sum_j w(i, j) \cdot V(s_j)$ where the mixing weights are determined by three continuous parameters:

$$\text{score}(i, j) = \alpha \cdot \text{content}(s_i, s_j) + \beta \cdot \text{displacement}(i - j), \quad w(i, j) = \text{softmax}(\text{score}) \cdot \text{connectivity}(i, j, r) \quad (4)$$

The parameters α (content dependence), β (displacement dependence), and r (connectivity radius) are continuous scalars. Four familiar architectures emerge as specific settings: CNN ($\alpha=0, \beta=1, r=3$), self-attention ($\alpha=1, \beta=0, r=\infty$), local attention ($\alpha=1, \beta=0, r=5$), and relative attention ($\alpha=1, \beta=1, r=\infty$).

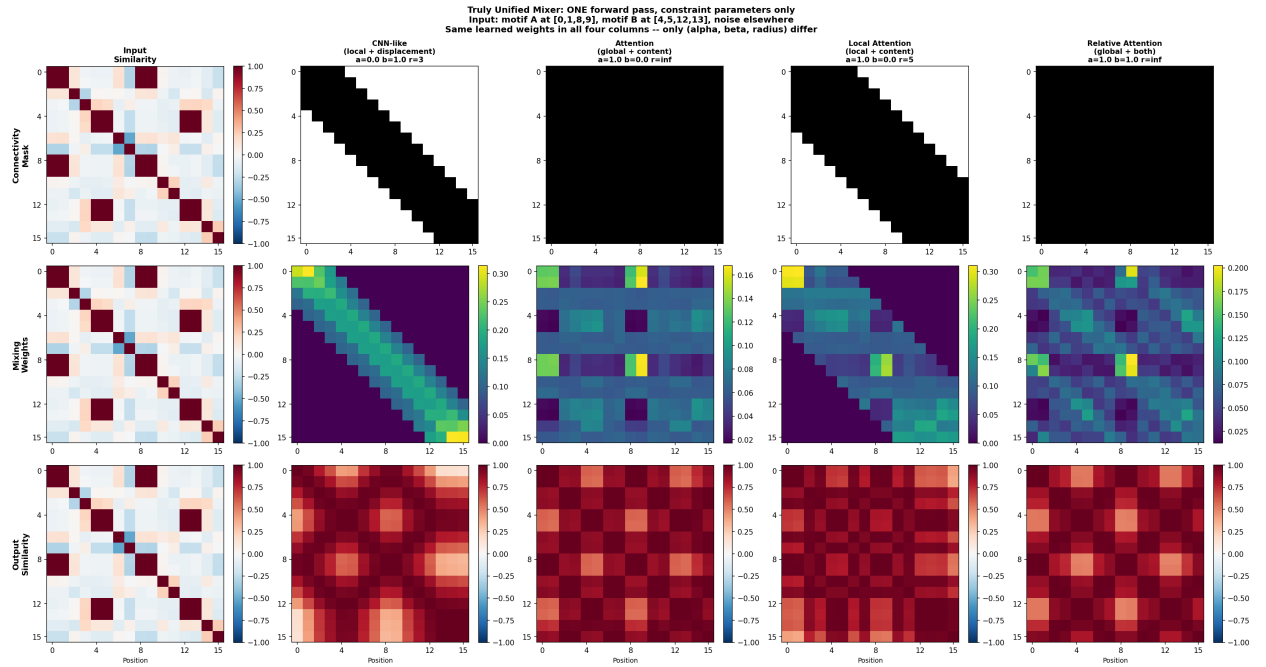


Figure 1: A single forward pass through one mixer instance with shared learned parameters, varying only the constraint parameters (α, β, r) . Row 1: connectivity masks. Row 2: learned mixing weights after softmax. Row 3: output cosine similarity. CNN produces banded Toeplitz structure (local, translation-invariant). Attention produces global content-similarity clusters. The same weights, the same code path—only the constraints differ.

Figure 1 shows the key result: one forward pass with no branching produces four distinct architectural behaviors. The CNN mixing weights form a banded Toeplitz structure (translation-invariant, local). The attention weights reflect global content similarity—positions sharing motif A attend to each other regardless of spatial distance. The output similarity matrices confirm the

behavioral consequences: CNN preserves local structure while attention creates long-range similarity clusters aligned with content. The “architecture” is not the code; it is the geometric prior imposed by the constraint parameters.

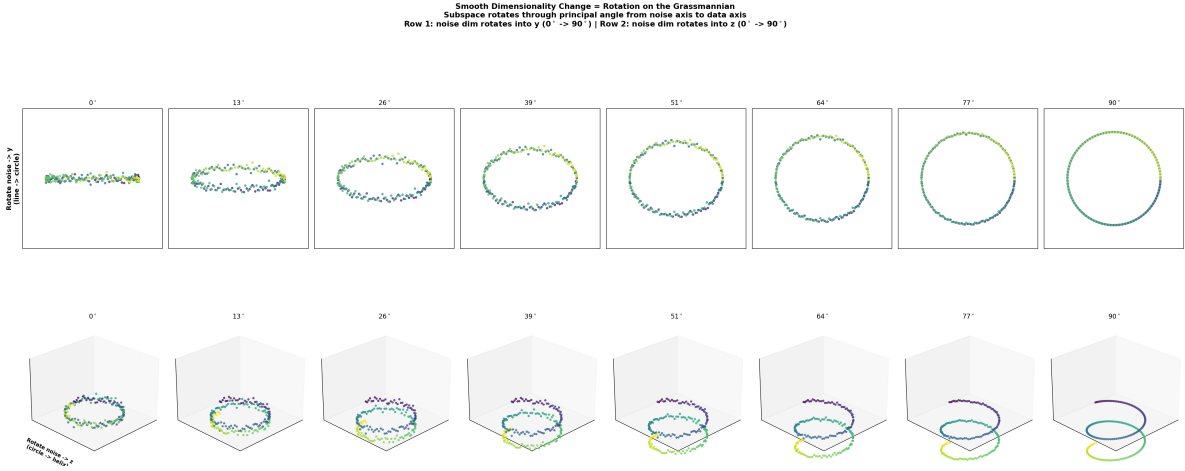


Figure 2: Smooth dimensionality growth as Grassmannian geodesic. Top row: a 2-plane rotates from $\{x, \text{noise}\}$ to $\{x, y\}$; the projected data transforms from a scattered line (0°) through intermediate ellipses to a clean circle (90°). Bottom row: a 3-volume rotates to reveal the helical height dimension.

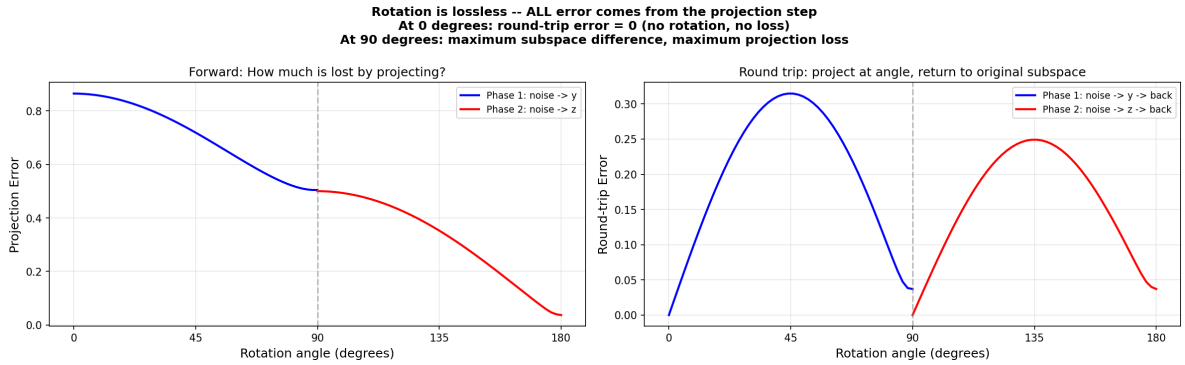


Figure 3: Rotation is lossless; all error comes from projection. Left: projection error decreases as the subspace rotates from noise toward signal axes. Right: round-trip error peaks at intermediate angles (near 45°) and drops at both 0° (no rotation, no loss) and 90° (fully orthogonal subspace—a clean basis swap). Information loss is architectural (the commitment to a subspace), not geometric (the rotation itself).

Figure 2 complements the mixer result by visualizing the extrinsic mode: as a subspace rotates on the Grassmannian, hidden structure in the data is continuously revealed. One dimension sees only a line; two reveal the circle; three reveal the full helical winding. The rotation is smooth—topological structure emerges progressively, not abruptly. Figure 3 quantifies the critical distinction: Grassmannian rotation is lossless; all information loss comes from the projection step. This has a direct architectural interpretation: a layer that changes its representational subspace (rotation on the Grassmannian) preserves all information. Loss occurs only when the network commits to a particular subspace and discards the orthogonal complement.

A second observation from the error curves: round-trip error peaks at intermediate rotation

angles (near 45°) and is minimal at both 0° (no rotation) and 90° (fully orthogonal subspace). Partial rotations are the worst regime—they mix signal and noise dimensions without cleanly separating them. This may explain a persistent architectural convention: layer width changes in practice almost always double or halve ($64 \rightarrow 128 \rightarrow 256$, or the FFN $4\times$ expansion in transformers [Vaswani et al., 2017]) rather than using fractional multipliers like $1.3\times$. A $2\times$ width expansion embeds into dimensions orthogonal to the original—a near- 90° Grassmannian rotation where information preservation is maximal. A fractional expansion would place the new subspace at an intermediate angle, in the high round-trip-error regime. The convention of power-of-two width changes is not merely computational convenience; it selects for Grassmannian rotations near 0° or 90° where the error landscape is flattest. The same logic applies to residual connections: by adding the identity (0° rotation) to the layer’s output, the skip connection anchors one component at the zero-loss point while permitting the learned transformation to make a full orthogonal jump.

6.2 Subspace Misalignment

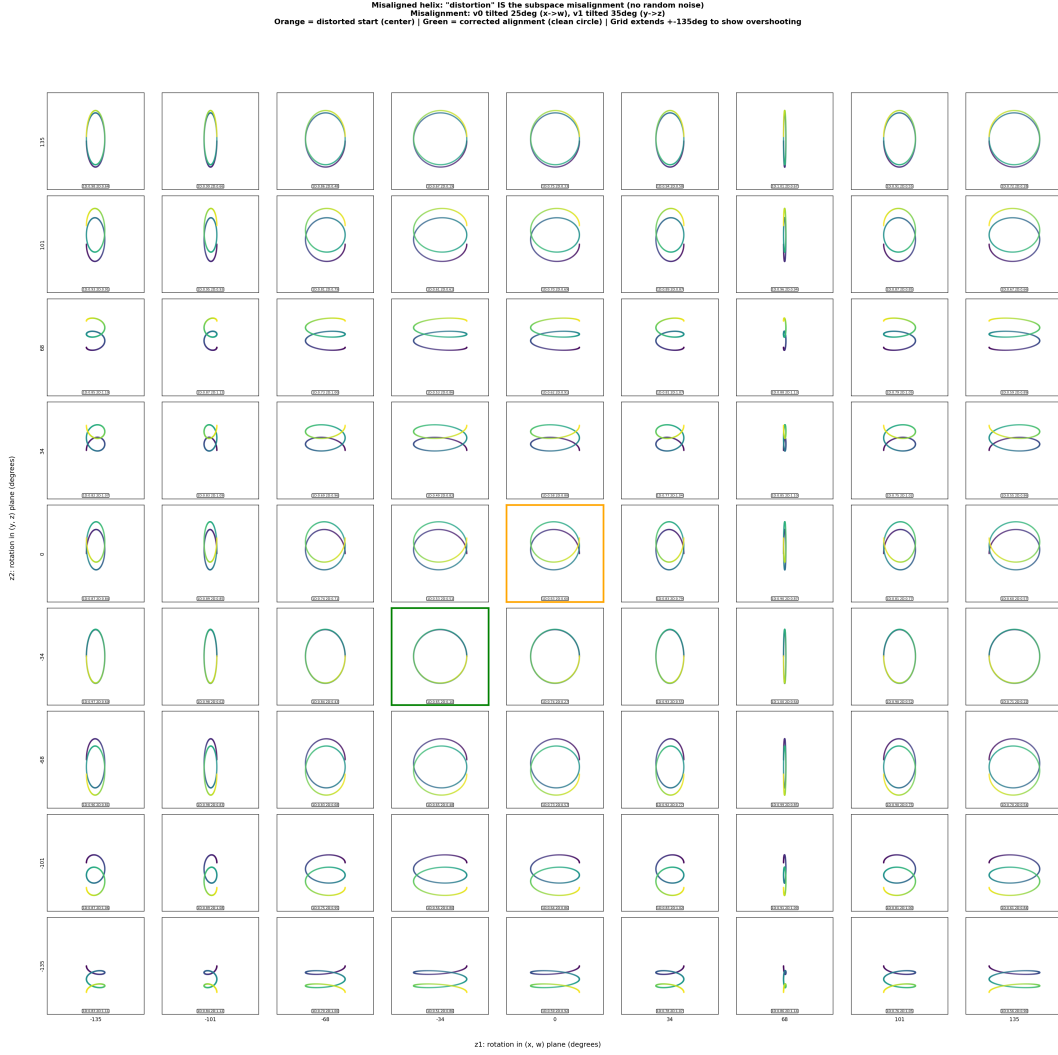


Figure 4: Subspace misalignment as Grassmannian error landscape. A clean helix viewed through a misaligned 2D subspace (orange border, center) with two independent misalignment angles z_1 (25° tilt in the x - w plane) and z_2 (35° tilt in the y - z plane). The green-bordered cell shows the correction point where both misalignments are exactly undone. Along the $z_1 \approx 68^\circ$ column, the shapes collapse to lines regardless of z_2 —misalignment in one Grassmannian direction renders misalignment in another direction undetectable from the projected data alone.

The misalignment experiment (Figure 4; see also Figure 16 in Appendix B) addresses the latent space interpretation directly. The “distortion” is not additive noise but subspace misalignment: the observation basis is tilted relative to the signal axes. This is exactly the form of corruption that Grassmannian rotation can correct—and it is the architectural analogue of geometric mismatch. The error heatmap shows that correcting a 2D representation requires simultaneously adjusting both Grassmannian angles; correcting only one leaves residual distortion that cannot be diagnosed from within the projected view. In the language of the framework: when intermediate representations are expressed in a suboptimal basis (features partially mixed with irrelevant dimensions), a learned

Grassmannian rotation can realign them, but the correction requires exploring the full Grassmannian neighborhood, not merely inspecting the current subspace.

6.3 Empirical Validation on MNIST

To test the framework’s predictions quantitatively, we train primitive-based architectures on MNIST (60K training / 10K test, 15 epochs, Adam with $\text{lr} = 0.001$, batch size 64, gradient clipping at $\|\cdot\|_{\max} = 1.0$). All models use depth 5 unless noted. Each primitive layer applies an $m=2$ (pairwise) mixer parameterized as $M = \exp(A)$ with either data-dependent (DD) or fixed parameters, followed by butterfly/bit-reversal/stride/cyclic routing permutations.

Test 1: The matrix exponential is the nonlinearity. We compare data-dependent mixers ($M = \exp(A(x))$, where A depends on input) against fixed mixers ($M = \exp(A)$ with learned but input-independent A), each with and without explicit ReLU activation. Results (Table 4):

Table 4: Activation boundary test. DD = data-dependent mixer; Fixed = input-independent mixer. All models use depth 5.

Configuration	Accuracy	Parameters
DD, no activation	79.10%	1345
DD + ReLU	80.98%	1345
DD + hidden layer + ReLU	83.75%	1763
Fixed, no activation	59.93%	265
Fixed + ReLU	59.52%	265
Fixed + hidden layer + ReLU	63.97%	683

The data-dependent mixer without any activation function achieves 79.10%—a 19.17 percentage point advantage over the fixed mixer (59.93%). Adding explicit ReLU to the fixed mixer recovers only 4 points (to 59.52%, within noise), while the DD mixer already provides strong nonlinearity through $\exp(A(x))$. The matrix exponential, when its argument depends on data, *is* the nonlinearity.

Test 2: Constraint space interpretation. We compare primitives (depth 3, 823 params) against CNNs at matched parameter budget: a 1-layer CNN with global average pooling (810 params) and a 2-layer CNN (754 params).

Table 5: Primitives vs. CNN at matched parameter count.

Configuration	Accuracy	Parameters
Primitives (depth 3)	76.34%	823
CNN 1-layer + GAP	46.03%	810
CNN 2-layer	82.20%	754

Primitives outperform the 1-layer CNN by 30 percentage points because the single convolution layer lacks sufficient receptive field for MNIST digits. The 2-layer CNN recovers and surpasses primitives by 5.86% because its spatial prior is well-matched to MNIST’s local structure. This illustrates the framework’s claim: CNN behavior exists as a point in the continuous constraint space of the same primitives. When that constraint point matches the data geometry (as 2D spatial locality matches handwritten digits), the constrained model outperforms the unconstrained one.

Test 3: Nonlinearity beyond parameter count. To confirm that the data-dependent mixer’s advantage comes from nonlinearity rather than parameter count, we compare against a fixed mixer with *more* parameters obtained by lifting to higher working dimension.

Table 6: Nonlinearity vs. capacity. Mean $\pm$ std over 3 seeds.

Configuration	Accuracy	Std	Parameters
Fixed shared ($d=3$)	59.95%	0.44%	265
Fixed shared (lifted $d=8$)	70.44%	0.14%	1714
Data-dependent shared ($d=3$)	78.79%	2.62%	1345

The fixed-lifted model has 27% more parameters (1714 vs. 1345) yet loses to the data-dependent model by 8.35 percentage points. Capacity alone cannot close the gap—nonlinearity from $\exp(A(x))$ genuinely matters. The higher variance of the DD model (2.62% vs. 0.14%) reflects greater expressive power coupled with initialization sensitivity.

Test 4: Dimension as geometric structure. We sweep the working dimension d to test the Grassmannian prediction that reducing dimensionality below the data’s intrinsic rank causes *structural* collapse, not merely reduced capacity.

Table 7: Lifting dimension sweep. Mean $\pm$ std over 3 seeds.

Configuration	Accuracy	Std	Parameters
$d=2$ (antilift)	48.46%	15.24%	456
$d=3$ (no lift)	80.82%	0.34%	1345
$d=4$ (mild lift)	87.07%	0.81%	3022

At $d=2$, both accuracy and training stability collapse catastrophically—15.24% standard deviation across seeds, with some runs failing entirely. This matches the Grassmannian prediction: losing a dimension is geometric information loss (the subspace can no longer span the data’s structure), not just fewer parameters. The transition from $d=3$ to $d=4$ is smooth (+6.25% with stable training), consistent with adding geometric headroom rather than crossing a structural threshold. The instability at $d=2$ mirrors the structural collapse observed in the Grassmannian geodesic experiment (Section 6) when rotating to intermediate angles on $\text{Gr}(k, D)$.

Test 5: Phase decomposition—not just nonlinearity, but full complex nonlinearity. Tests 1–3 establish that nonlinearity from $\exp(A(x))$ is essential: it subsumes explicit activations, outperforms fixed mixers, and cannot be replicated by increased linear capacity. But the $\exp(A)$ parameterization contains finer structure. Any real matrix A decomposes uniquely as $A = S + K$ where $S = (A + A^\top)/2$ is symmetric and $K = (A - A^\top)/2$ is skew-symmetric. Symmetric A has real eigenvalues, so $\exp(S)$ produces pure magnitude scaling (stretching and compressing along eigenvector directions). Skew-symmetric A has purely imaginary eigenvalues $\pm i\beta$, so $\exp(K)$ produces pure rotation (orthogonal matrix, norm-preserving)—native sinusoidal structure via Euler’s formula. The full $\exp(A)$ accesses both simultaneously, with the network learning what mix of scaling and rotation the task requires. The question is whether the full complex structure is necessary, or whether one mode alone suffices.

We test this by constraining A to symmetric-only ($\exp(S)$, real eigenvalues, magnitude scaling), skew-symmetric-only ($\exp(K)$, imaginary eigenvalues, rotation), and unconstrained ($\exp(A)$, both),

alongside a GELU baseline replacing $\exp(A)$ with $I + \text{GELU}(A)$. On $\sin(x)$ regression over $[0, 2\pi]$ (500 epochs, 3 seeds, depth swept 1–8):

Table 8: Phase decomposition on $\sin(x)$ approximation (test MSE).

Layers	GELU	Symmetric	Skew-sym	Full $\exp(A)$
1	0.4105	0.6741	1.4423	0.1000
2	0.3530	0.0039	0.0022	0.0005
4	0.2773	0.0007	0.0015	0.0001
8	0.1406	0.0001	0.0002	0.0001

Full $\exp(A)$ reaches 10^{-4} MSE at 4 layers while GELU remains at 1.4×10^{-1} at 8 layers—three orders of magnitude worse. This is not a marginal improvement but a qualitative difference: GELU’s elementwise nonlinearity cannot efficiently represent periodic structure, while $\exp(A)$ ’s Lie group structure provides native access to rotation.

On MNIST classification (5 layers, 20×20 center crop to eliminate dead-pixel pairs from the border, 3 seeds), the picture is different:

Table 9: Phase decomposition on MNIST classification (test accuracy %).

Condition	Mean Acc. (%)	Std	Individual Runs
GELU ($I + \text{GELU}(A)$)	77.62	2.45	74.72, 79.17, 78.97
Symmetric $\exp(S)$	73.16	0.36	72.94, 73.59, 72.95
Skew-symmetric $\exp(K)$	73.28	1.84	74.42, 71.05, 74.37
Full $\exp(A)$	77.87	1.51	75.76, 78.67, 79.18

Full $\exp(A)$ matches GELU rather than exceeding it, in contrast to the $\sin(x)$ result. Two factors explain the difference. First, 81.4% of pixel values in MNIST are zero; the shared mixer is pulled toward identity by dead-pixel pairs, diluting useful gradient approximately 2:1. The center crop from 28×28 to 20×20 reduces this from 45% to 38.5% dead pairs but does not eliminate it. Second, MNIST classification does not require the oscillatory rotational structure where $\exp(A)$ excels—the task is predominantly magnitude-based (which features are present), not phase-based (periodic structure). Both constrained conditions (symmetric-only and skew-symmetric-only) fall approximately 4.5 percentage points below the full and GELU baselines, confirming that both scaling and rotational modes contribute to classification performance even when neither mode alone is sufficient to match the unconstrained parameterization.

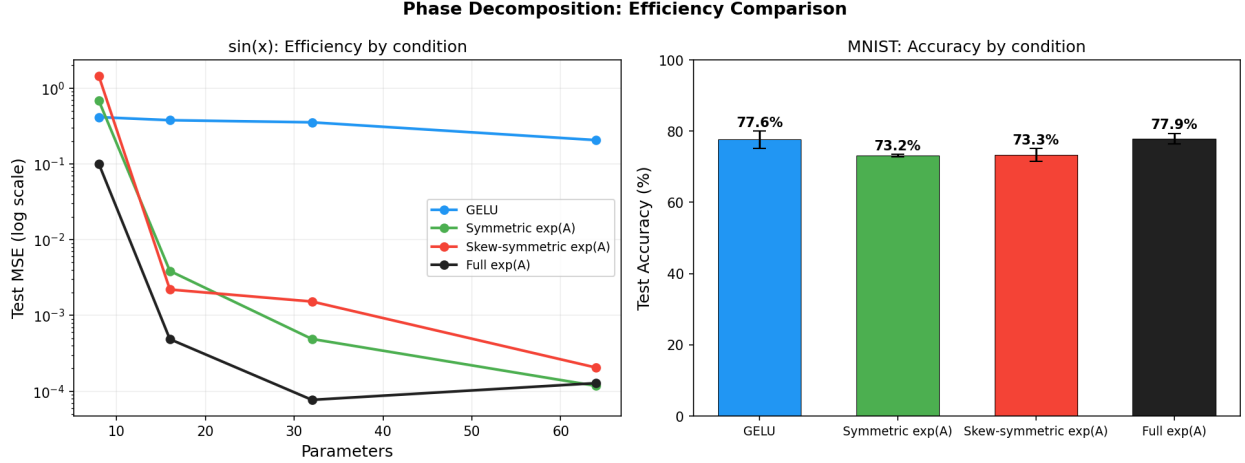


Figure 5: Phase decomposition efficiency comparison across both tasks. Left: $\sin(x)$ test MSE vs. parameter count (log scale), showing full $\exp(A)$ reaching 10^{-4} while GELU plateaus above 10^{-1} . Right: MNIST test accuracy by condition, showing full $\exp(A)$ and GELU tied at $\sim 78\%$ with constrained conditions ~ 4.5 pp lower.

Methodological note. The GELU baseline uses $M = I + \text{GELU}(A)$ rather than $M = \text{GELU}(A)$, since $\text{GELU}(0) = 0$ produces an all-zero mixer at initialization that blocks information flow. The identity-plus formulation matches the $\exp(0) = I$ starting point of the other conditions, ensuring a fair comparison. The center crop eliminates border pixels that are always zero across the entire MNIST dataset, which would otherwise contribute zero-gradient pairs that dilute the mixer’s learning signal.

Per-layer activation diversity. Plotting the effective scalar activation (first output component vs. scalar input x) at each mixer layer reveals that $\exp(A)$ does not learn a single activation function applied uniformly. Each layer develops a qualitatively different nonlinearity (Figure 6). On $\sin(x)$, early layers learn coarse shaping (moderate S-curves), middle layers develop oscillatory structure (the rotational mode contributing periodicity), and later layers perform fine adjustment (nearly linear corrections). The skew-symmetric condition shows native sinusoidal character at each layer, while the symmetric condition develops smooth monotone scaling—confirming that the two components of $A = S + K$ provide distinct nonlinear modes.

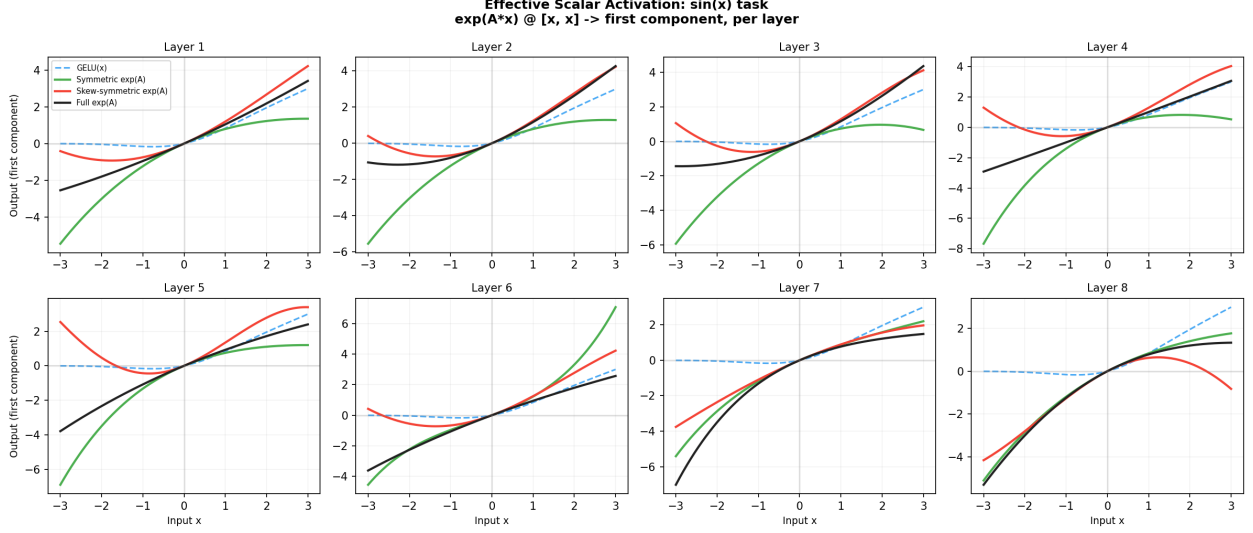


Figure 6: Effective scalar activation per mixer layer on $\sin(x)$ regression (8 layers). Each layer learns a different nonlinear function. GELU (blue dashed) remains approximately linear throughout; skew-symmetric (red) develops oscillatory structure; full $\exp(A)$ (black) combines both modes and adapts per layer.

On MNIST classification (Figure 7), the per-layer diversity is even more pronounced: early layers show moderate nonlinearity, while deeper layers develop increasingly extreme scaling (layer 5 reaches output magnitudes of 10^3). This depth-dependent amplification is the eigenvalue divergence predicted by the gradient analysis of $\exp(A)$: when eigenvalues of A grow during training, $\exp(\lambda)$ amplifies exponentially, producing gradient explosion in large-eigenvalue directions and collapse in small-eigenvalue directions. Averaging the effective activation across all layers and spatial positions cancels the interior structure entirely, preserving only the extreme-input tails where exponential growth dominates regardless of the learned A —confirming that the per-layer specialization is not noise but the network exploiting different nonlinear regimes at different depths.

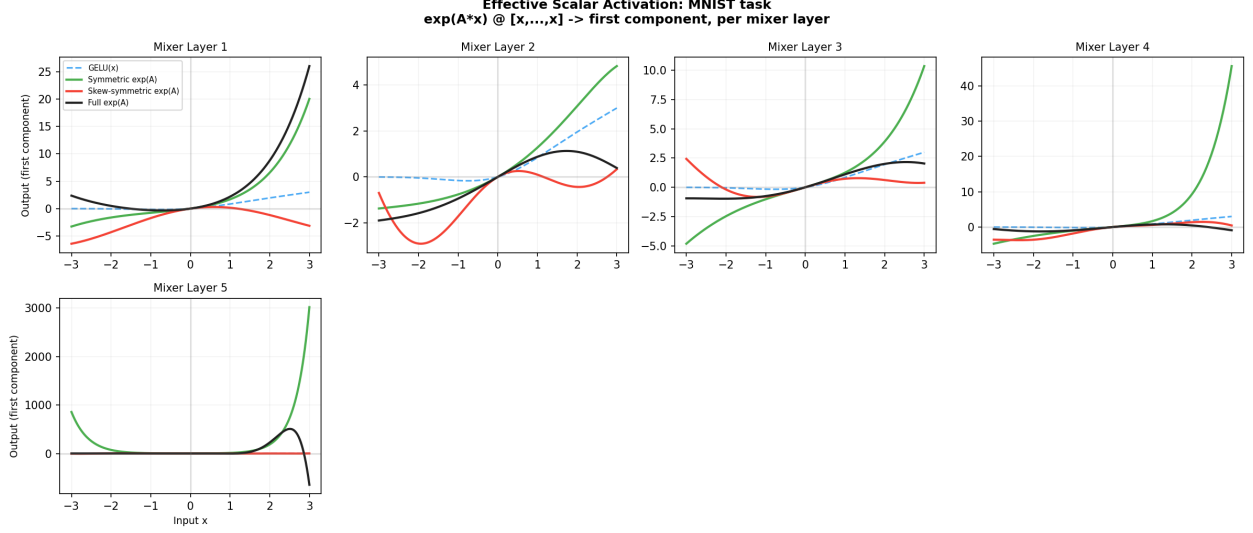


Figure 7: Effective scalar activation per mixer layer on MNIST (5 layers, 20×20 center crop). Each layer learns a different effective nonlinearity. Layer 5 develops extreme amplification (note scale change), reflecting eigenvalue divergence at depth.

Eigenvalue migration. Tracking the eigenvalues of the learned A matrices during training on $\sin(x)$ (full $\exp(A)$, 8 layers, Figure 17 in Appendix B) reveals active development of rotational structure: the proportion of complex eigenvalues rises from 4/16 at initialization to 12/16 during early exploration (epoch 50), then converges to 6/16—the three conjugate pairs needed to represent a single sinusoid. The imaginary components grow from 0.04 to 0.15 over training, confirming that the network discovers rotational structure from data rather than having it imposed.

Spatially varying eigenvalue structure. On MNIST, eigenvalue scatter plots sampled across a 4×4 grid of positions in the 20×20 center crop (Figure 8) reveal that the data-dependent A learns different eigenvalue configurations at different spatial locations. All positions develop complex conjugate pairs (points symmetric about the real axis), but the distribution varies: some positions show eigenvalues clustered near the origin (low-activity regions where the mixer acts near-identity), while others develop larger imaginary components (high-activity regions where the mixer performs substantive rotation). This is the framework’s prediction realized: the primitive is not learning edge detectors (as a convolution would) but learning position-dependent complex rotations—full circles in the eigenvalue plane rather than real-axis-only magnitude scaling. The complex eigenvalue structure at each position reflects the local geometric requirements of the data, with the full $\exp(A)$ accessing both rotational and scaling modes simultaneously.

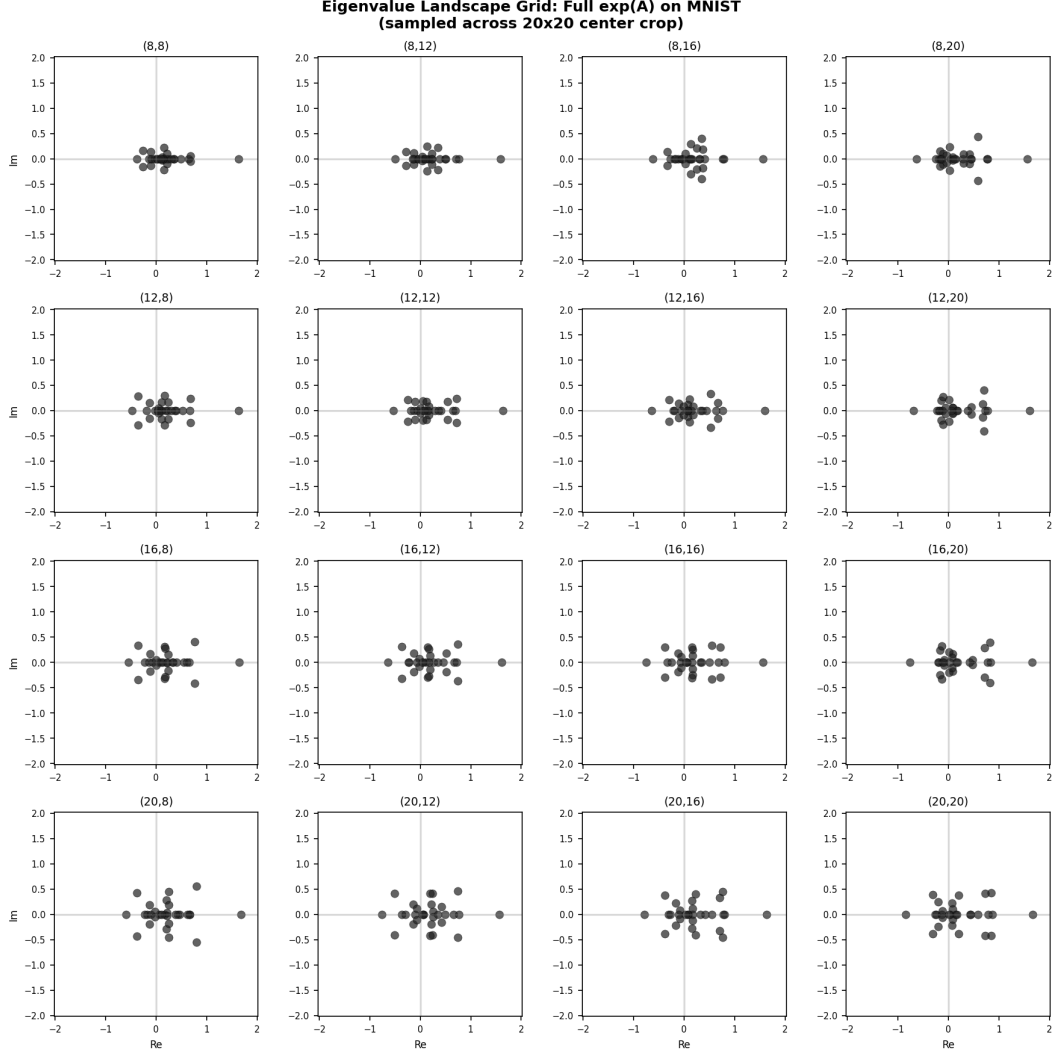


Figure 8: Eigenvalue landscape of the learned A across a 4×4 grid of spatial positions on MNIST (full $\exp(A)$, 20×20 crop). Complex conjugate pairs appear at all positions, with the distribution varying spatially—the mixer learns position-dependent rotational structure rather than a uniform activation.

GELU as frozen $\exp(A)$. The relationship between GELU and $\exp(A)$ is clarified by the per-layer results. GELU was derived by Hendrycks and Gimpel [Hendrycks & Gimpel, 2016] as the expected value of a stochastic gating process under Gaussian input assumptions: $\text{GELU}(x) = x \cdot \Phi(x)$, where Φ is the Gaussian CDF. It is a single fixed function applied identically at every layer and every position. The $\exp(A)$ parameterization subsumes GELU as one point in a much larger space of per-layer, per-position adaptive nonlinearities. GELU cannot vary with depth; $\exp(A)$ learns different effective activations at each layer (coarse shaping early, fine adjustment late, extreme amplification at depth). GELU cannot vary with spatial position; $\exp(A)$ learns position-dependent eigenvalue structure (the grid in Figure 8). The three-order-of-magnitude gap on $\sin(x)$ reflects this fundamental asymmetry: periodic structure requires the rotational mode that GELU’s fixed Gaussian-CDF shape cannot provide, while $\exp(A)$ discovers it automatically through complex eigenvalue development (see Figure 18 in Appendix B for a cross-task eigenvalue comparison).

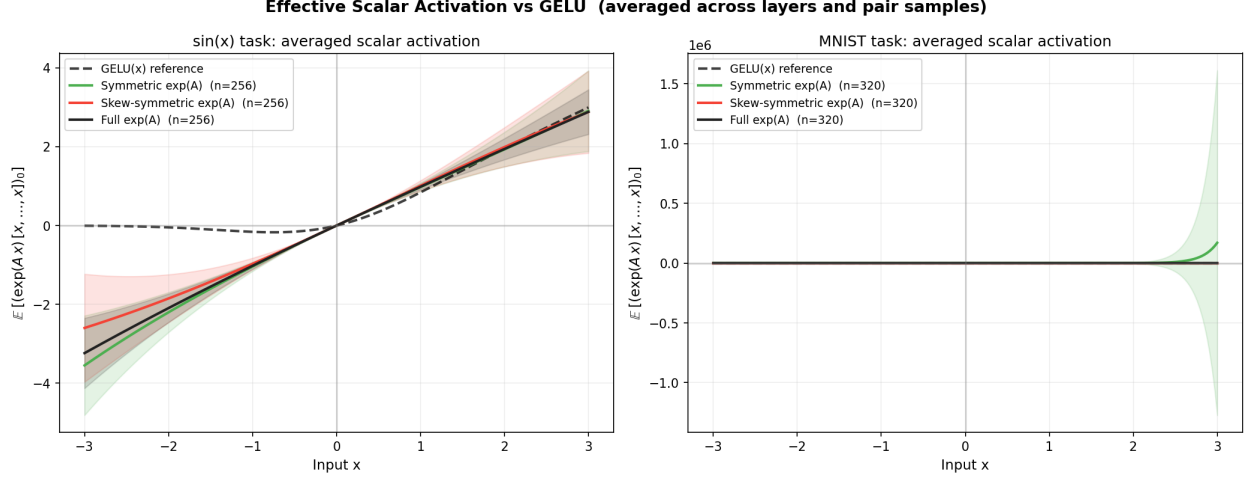


Figure 9: Effective scalar activation (averaged across layers and pair positions) for each condition vs. GELU reference. Left: $\sin(x)$ task, where symmetric (green) learns smooth monotone scaling, skew-symmetric (red) develops oscillatory structure, and full $\exp(A)$ (black) combines both. Right: MNIST task, where full $\exp(A)$ develops extreme amplification at large input magnitudes (note the 10^6 scale), reflecting eigenvalue divergence at depth. The GELU reference (blue dashed) is nearly linear in both plots, unable to access either the oscillatory or extreme-amplification regimes.

Gradient instability at depth. The extreme amplification at layer 5 in Figure 7 is not a training failure but a consequence of $\exp(A)$ ’s gradient structure. Standard backpropagation through $\exp(A)$ produces gradients involving $(\exp(\lambda_k) - \exp(\lambda_l))/(\lambda_k - \lambda_l)$, where λ_k, λ_l are eigenvalues. When eigenvalues diverge during training, gradients explode in large-eigenvalue directions and collapse in small-eigenvalue directions. This is a loss-architecture mismatch: the standard task loss (MSE or cross-entropy) is unbounded in eigenvalue space, while the geometric loss $\sin^2(\theta)$ is bounded $[0, 1]$ regardless of eigenvalue magnitude. The per-layer activation plots make this visible—the deeper layers, where eigenvalues have had more training steps to grow, show the most extreme amplification. Geometric regularization (adding a $\sin^2(\theta)$ penalty to the training loss) should stabilize this by providing bounded gradients, though we leave this to future work.

Eigenvalue stability of $\exp(A)$. Training the data-dependent $\exp(A)$ mixer with standard cross-entropy loss produces a characteristic instability: after 20–30 epochs of stable learning (reaching 95% accuracy on MNIST classification), eigenvalues of A escape to large magnitude and $\exp(\lambda)$ overflows, producing NaN gradients. For symmetric A (real eigenvalues), the mechanism is straightforward—the gradient of the matrix exponential involves terms $(\exp(\lambda_k) - \exp(\lambda_l))/(\lambda_k - \lambda_l)$, which diverge when eigenvalues spread apart. The natural remedy exploits the structure of symmetric A directly: since $A = Q\Lambda Q^\top$ with real Λ , we compute $\exp(A) = Q \text{diag}(\exp(\text{clamp}(\lambda_i, -c, c))) Q^\top$ via eigen-decomposition rather than Padé approximation. With $c = 3$ (bounding the per-eigenvalue scaling ratio to $\exp(3) \approx 20\times$), training is stable through 100 epochs with no accuracy loss. This spectral clamp acts as an implicit Frobenius-type constraint on the Lie algebra element A , bounding the operator norm $\|\exp(A)\| \leq \exp(c)$ without adding an explicit penalty term to the loss. For skew-symmetric A , the unitarity of $\exp(K)$ ($|\exp(i\theta)| = 1$) suggests rotation is intrinsically bounded, but this has not been empirically verified at scale.

Summary of MNIST findings. These five tests support the framework’s core claims: (1) the matrix exponential parameterization provides intrinsic nonlinearity that subsumes explicit activation functions; (2) CNN and primitive performance differences reflect the match between geometric prior and data structure, not fundamental architectural superiority; (3) nonlinearity from data-dependent $\exp(A)$ cannot be replicated by increased linear capacity; (4) working dimension has a geometric interpretation on the Grassmannian, with structural collapse below the data’s intrinsic rank; and (5) the full complex structure of $\exp(A)$ —both scaling and rotational modes—is required, as restricting to either mode alone costs ~ 4.5 percentage points on MNIST and three orders of magnitude on periodic tasks, demonstrating that the relevant nonlinearity is not merely “any nonlinearity” but the specific Lie group structure of the matrix exponential. Additional results (pixel ordering invariance, receptive field analysis) appear in Appendix A.

7 Geometric Mismatch: From Representation to Optimization

The three primitives define *what* a network can compute; this section addresses *how well* that computation matches the data’s geometry, how to diagnose the gap, and how to close it. We begin by defining the target (meaning), characterize the gap (latent space as mismatch), decompose it (parallel vs. perpendicular content), formalize three independent diagnostics, and then show that the $GL(D)$ view yields differentiable probes for each.

7.1 Meaning as Hidden Metadata

The framework requires a precise definition of “meaning space.” A task selects which subset of meaning to extract, but meaning itself exists independently: a vector of accelerator telemetry does not label itself “running normally,” yet that description is recoverable by the right transformation. Meaning is the hidden descriptive layer—the categories, states, and relationships that the data is an expression of but does not self-report. The meaning subspace is the quotient of data space by variation that does not change the underlying description, where the equivalence classes are determined by the data’s generative structure rather than by an external task.

Data, meaning, and task output lie on a Grassmannian continuum. Data is a specific realization with all details; meaning is what is invariant across realizations; task output is a further projection selected for a particular purpose. The question of what occupies the remaining dimensions—the orthogonal complement of the meaning subspace—leads directly to the geometric interpretation of latent space.

7.2 Latent Space as Geometric Mismatch

The term “latent space” is typically treated as a mathematical construct—the hidden representation space a network learns. The Grassmannian framework suggests a different interpretation: latent space is not a space the data lives in, but a *measurement of the gap* between the data’s true manifold structure and the architecture’s imposed geometric prior. What makes features latent is mismatch. When the architecture’s constraints don’t permit the direct Grassmannian route from data to meaning, the intermediate representations are artifacts of the detour—they encode the data in a basis that reflects the architecture’s limitations rather than the data’s structure. A fully matched architecture would not need a latent space; the data’s geometry would flow directly through the constraints to the output.

Block structure. The Grassmannian geodesic provides a concrete realization. Consider a representation in $\mathbb{R}^{2k}$ decomposed into two k -dimensional subspaces, with the block rotation

$$\begin{pmatrix} \cos(\theta)I_k & -\sin(\theta)I_k \\ \sin(\theta)I_k & \cos(\theta)I_k \end{pmatrix} \begin{pmatrix} \mathbf{u} \\ \mathbf{v} \end{pmatrix}$$

parameterizing a geodesic on $\text{Gr}(k, 2k)$. The diagonal blocks correspond to “readable” subspaces—input and output representations aligned with the readout frame. The off-diagonal blocks are the latent space: content rotated away from both bases and inaccessible to any linear probe aligned with either. At $\theta = 0$, all content is readable in the input frame; at $\theta = \pi/2$, fully readable in the output frame; at $\theta = \pi/4$, maximally latent—maximum $\sin^2(\theta)$ error, minimum interpretability.

This gives a measurable *latent space occupancy* at each layer: the fraction of total energy in the off-diagonal blocks, $(\|\mathbf{B}\|^2 + \|\mathbf{C}\|^2)/(\|\mathbf{A}\|^2 + \|\mathbf{B}\|^2 + \|\mathbf{C}\|^2 + \|\mathbf{D}\|^2)$. A well-matched architecture minimizes this quantity at every layer; a poorly matched one forces content through extended latent detours.

7.3 Decomposing Latent Content

Not all content in the off-diagonal blocks is heading to the same destination. At any layer ℓ , the representation occupies $V(\ell) \in \text{Gr}(k, 2k)$. The output subspace V_{out} is another point on the same Grassmannian. The Riemannian logarithm $\text{Log}_{V(\ell)}(V_{\text{out}})$ yields a tangent vector—the direction and magnitude of the geodesic from the current representation to meaning. This decomposes the off-diagonal content into two orthogonal components:

1. **Parallel to the geodesic tangent** $\rightarrow$ *meaning in transit*. Content on the direct Grassmannian path from $V(\ell)$ to V_{out} . It is in the off-diagonal blocks because the geodesic requires intermediate steps, but every unit of energy here is productive.
2. **Perpendicular to the geodesic tangent** $\rightarrow$ *task-irrelevant meaning*. Content rotated into the off-diagonal blocks but not heading toward V_{out} . This is not noise—it is meaning that the current task does not require. The same component may be parallel to a *different* task’s geodesic.

The **meaning fraction** $\mu_\ell = \|c_{\parallel}\|^2/\|c_{\text{off}}\|^2$ quantifies task efficiency at each layer. Because μ_ℓ depends on the choice of V_{out} , the same layer can have high meaning fraction for one task and low for another—the representation is task-aligned or task-misaligned, not intrinsically good or bad. This is computable for any trained network without retraining: extract the subspace at each layer, compute the Riemannian log map, and project [Bendokat et al., 2024].

Resolving the representation learning debate. The meaning fraction resolves a longstanding tension. One tradition [Bengio et al., 2013] treats latent space as *discovering* meaningful structure; another [Tishby et al., 2000] treats it as *compressing* away irrelevance. Both assume a single partition into meaningful and meaningless content. The Grassmannian decomposition shows the partition is task-relative: the parallel component is what both traditions isolate for a given task; the perpendicular component is not waste but a reservoir of alternative meanings—task-irrelevant for the current objective but potentially essential for others.

Transfer learning as perpendicular reuse. Transfer learning succeeds because the perpendicular components are alternative meaning, not noise. A model pretrained on ImageNet populates its off-diagonal blocks with meaning directions for thousands of visual concepts. Fine-tuning on a medical imaging task finds that components perpendicular to ImageNet’s V_{out} may be parallel to the new task’s V_{out} . Large pretrained models transfer well because they have explored many geodesic directions; narrow models have high μ_ℓ for one task but an impoverished perpendicular reservoir. The disentanglement literature— β -VAE [Higgins et al., 2017], FactorVAE, DCI metrics—attempts to axis-align the parallel directions for all tasks simultaneously, a challenge that reflects the difficulty of matching the architectural prior to all downstream generative structure at once.

Design criterion. For a *task-specific* architecture, maximize μ_ℓ —concentrate rotation along the geodesic to the known V_{out} . For a *general-purpose* architecture, maintain a rich perpendicular reservoir, accepting lower μ_ℓ for any single task in exchange for transferability across many.

7.4 Three Diagnostic Loss Types

The meaning fraction decomposes off-diagonal content by *direction*; we now decompose architectural failure by *source*. Standard gradient-based optimization receives a single scalar loss and computes $\partial\mathcal{L}/\partial w$ for every weight. It does not know whether a parameter belongs to a permutation, a pointwise transform, or a mixer—it conflates three structurally distinct sources of error:

1. **Parameter loss** (standard): are the matrix entries correct? This is what Adam optimizes—it operates in parameter space and is agnostic to architecture.
2. **Geometric loss** (Grassmannian): is the subspace path efficient? Measured by $\sin^2(\theta_j)$ per principal angle—the same quantity that defines latent space occupancy above. This encompasses both embedding dimensionality (wrong rank forces subspace detours) and channel count (too few independent interaction metrics constrains the Grassmannian path). The meaning fraction μ_ℓ further decomposes geometric loss into its task-productive component (parallel) and its reservoir-maintenance component (perpendicular).
3. **Topological loss** (routing): can the required information flow happen at all? Measured by performance degradation as graph distance increases under a fixed routing topology.

These are not three optimization objectives but three diagnostics that decompose *where capacity goes*: a high topological loss means parameters are spent relaying information through intermediaries; a high geometric loss means parameters are spent recovering from subspace detours; a high parameter loss means the weights are genuinely undertrained. Only parameter loss is directly reduced by gradient descent. Topological loss requires architectural change. Geometric loss can be indirectly reduced by training but benefits from geometrically informed regularization or—as we show next—from targeted extrinsic probing.

7.5 Experimental Evidence

Topological distance. To isolate the permutation primitive’s contribution, we train an $m=2$ (pairwise) mixer at graph distance $d=1$ (tokens directly paired), freeze it, and evaluate at distances $d = 1$ through 16 on a 32-token addition task ($a + b$), varying only the routing permutation. The mixer is a fixed $\exp(A)$ matrix (not data-dependent), eliminating all nonlinearity so that the pipeline is linear in the input token values.

Table 10: Topological loss: addition error vs. routing distance (preliminary, 5 seeds, 10 random position pairs per seed).

Distance d	1	2	4	8	12	16
MSE	0.188	0.291	0.750	1.005	0.974	1.250
$\pm$	0.269	0.112	0.212	0.133	0.332	0.498

Error grows roughly linearly with distance, saturating near chance (~ 1.0 MSE) around $d = 6-8$, defining a *topological horizon* beyond which the relay chain cannot deliver useful signal within the available depth (Figure 19 in Appendix B). This is additive dilution, not exponential compounding: each relay hop adds approximately constant error as the signal mixes with intermediate token content.

Routing topology comparison. With a fixed token pair at positions $(0, 16)$ and identical mixer weights, bit-reversal routing (graph distance 2 for this pair) achieves $\sim 700\times$ lower error than identity routing (pair unreachable), despite having the second-worst total connectivity (only 4 of 32 nodes reachable from position 0). The winning mechanism is *shortcut*: a sparse topology that happens to directly connect the required pair outperforms a dense topology that diffuses signal across many irrelevant paths (Figure 10). Wrong routing is worse than no routing—wide reach without targeted connectivity dilutes the signal below the single-operand fallback.

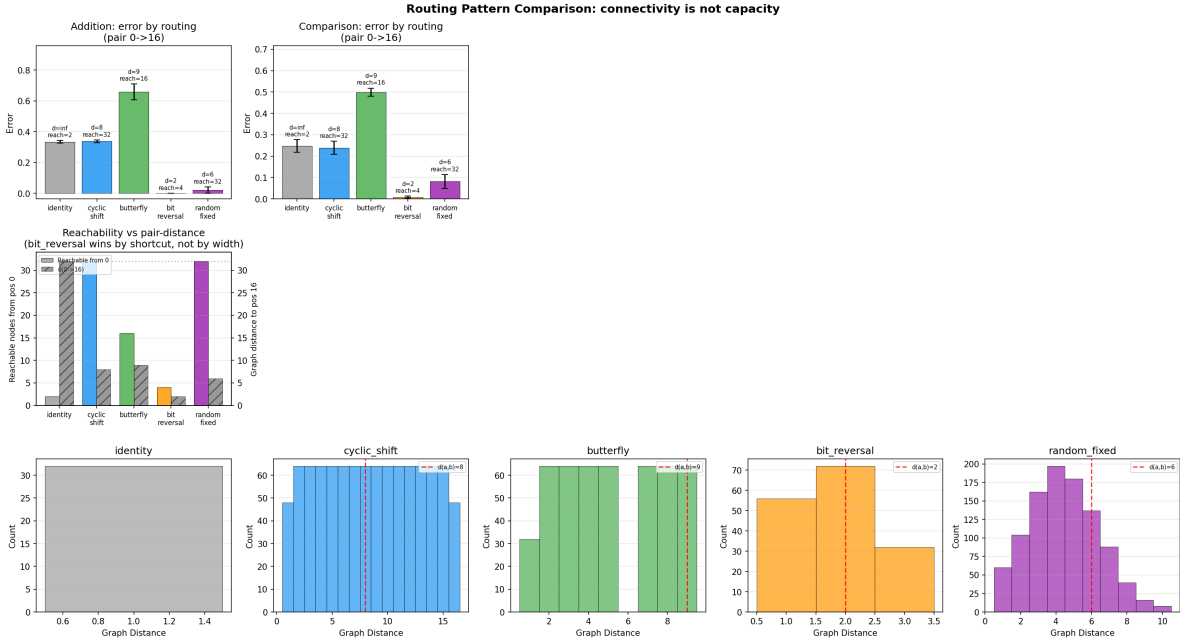


Figure 10: Routing topology comparison on fixed pair $(0, 16)$. Bit-reversal achieves $\sim 700\times$ lower error than identity on addition despite having the second-worst total connectivity. Error bars per routing, with reachability and pair-specific graph distance shown.

Information retention and phase transitions. Information retention (linear probe R^2 for recovering (a, b) from intermediate representations) shows a phase-transition structure at $L = d$: for $L < d$, $R^2 \approx 0$ at the target position (information has not yet arrived through the relay chain); at

$L = d$, a brief dip as the arriving signal undergoes geometric reorganization; for $L > d$, recovery to high R^2 as subsequent mixing layers render the combined signal linearly accessible. This transition from relay phase to processing phase is visible across all three diagnostic measurements (Figure 11) and suggests the losses operate sequentially rather than simultaneously—topological loss dominates layers 1 through d , geometric loss dominates layers $d+1$ through L .

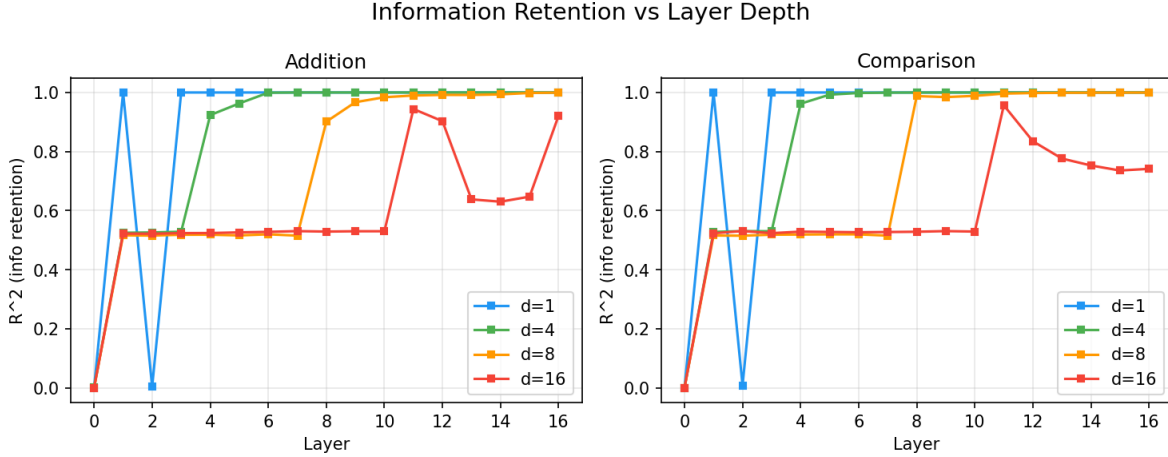


Figure 11: Information retention (linear probe R^2) for recovering operand values from intermediate representations at each layer depth, for relay distances $d = 1, 4, 8, 16$. A phase transition at $L = d$ marks the switch from relay-dominated to processing-dominated computation.

Per-primitive decomposition. Per-layer aggregate measurements of the three diagnostics show apparent coupling: varying routing distance simultaneously changes the Grassmannian rotation path because $\exp(A)$ is applied at every layer regardless of routing state. To determine whether this coupling is fundamental or a measurement artifact, we decompose each layer’s work into its two primitive steps. Every layer applies **permute** $\rightarrow$ **mix**. We define ΔT_ℓ as the permutation’s routing advancement (change in merge distance between target positions at layer ℓ) and G_ℓ as the mixer’s geometric action (rotation angle applied to the target-bearing pair at layer ℓ), then compute the Pearson correlation $r(\Delta T, G)$ across layers.

The correlation distribution is *bimodal*, not uniformly zero. At trivial relay distances ($d=1$) and full-chain relays ($d=16$), $|r| \approx 0$. At working relay distances ($d=4, 8$), correlations reach $r = 0.55$ – 0.82 on comparison, with the mixer rotating more on layers where routing has just advanced. Bit-reversal routing on comparison shows $r_b = -0.87$ —the mixer rotates *less* when routing advances and *more* when routing idles, reflecting the surplus depth (14 layers remaining after $d=2$ relay completion).

This pattern is *division of labor*, not loss coupling (Figure 12; see also Figure 20 in Appendix B). The three loss types measure independent dimensions of the problem—topological reachability, subspace alignment, parameter accuracy—but a trained model learns to *coordinate* its primitives across these dimensions. When the permutation delivers new information to a pair, the mixer has fresh material to process; when routing idles, the mixer works on previously delivered content. The correlation reflects efficient resource allocation by the optimizer, not fundamental dependence between the diagnostics. The coordinate axes are orthogonal even when the trained solution traces a diagonal path through the space they span.

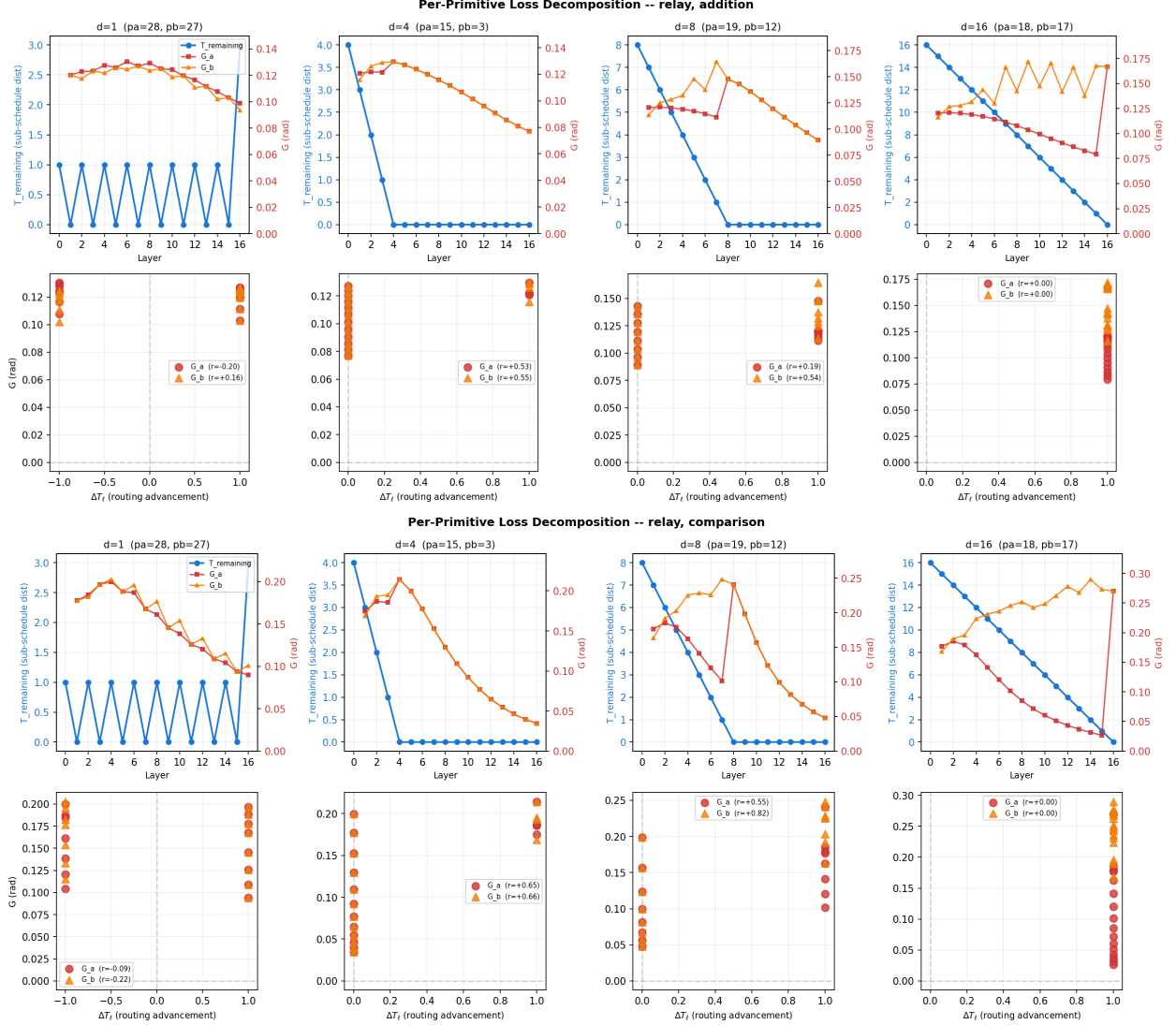


Figure 12: Per-primitive decomposition across relay distances ($d = 1, 4, 8, 16$). Top rows: per-layer Grassmannian angle G_ℓ (orange) and routing advancement ΔT_ℓ (blue). Bottom rows: scatter of geometric action vs. routing advancement with Pearson r . Addition (top) and comparison (bottom). Middle distances show coordinated primitive behavior ($r = 0.55$ – 0.82); trivial and full-chain distances show $r \approx 0$.

Grassmannian projection oscillation. The per-primitive traces reveal a characteristic alternating pattern: odd-numbered layers show higher geometric loss than even-numbered layers. This $2k$ periodicity (where k is the subspace dimension) arises from the Grassmannian geodesic structure. The projection operator $P_{V(t)} = V(t)V(t)^\top$ sweeps through $\text{Gr}(k, 2k)$ as layers accumulate, and the energy captured by a fixed readout frame oscillates as $\cos^2(n\theta)$ per principal angle, where θ is the rotation per layer [Bendokat et al., 2024]. Information is never lost (each $\exp(A)$ is invertible), but *accessibility* oscillates—the readout frame captures varying fractions of the total signal depending on the current rotation angle. The layers where the representation is “between” subspaces (near $\theta = \pi/4$) are maximally lossy, consistent with the $\sin^2(\theta)$ error established in Section 6.

7.6 Differentiable Extrinsic Probes

The diagnostics above identify what is wrong; the $\text{GL}(D)$ view of the full network suggests how to fix it. The gap between Adam’s single gradient signal and the three structural sources of error is precisely why architecture search exists as a separate problem from training. But it need not be. Consider a network with L blocks, each of dimension k , represented as a $D \times D$ matrix ($D = Lk$) with learned $k \times k$ blocks on the diagonal and zeros elsewhere. Each loss type corresponds to a structurally distinct perturbation of this matrix:

$$\underbrace{\begin{pmatrix} B_1 & & & \\ & B_2 & & \\ & & B_3 & \\ & & & B_4 \end{pmatrix}}_{\text{(a) Baseline}} \quad \underbrace{\begin{pmatrix} B_1 & & & \\ & B_2 & & \\ & & \boxed{B'_3} & \\ & & & B_4 \end{pmatrix}}_{\text{(b) Rank probe}} \quad (5)$$

$$\underbrace{\begin{pmatrix} B_1 & & \varepsilon I & \\ & B_2 & & \\ & & B_3 & \\ & & & B_4 \end{pmatrix}}_{\text{(c) Topology probe}} \quad \underbrace{\begin{pmatrix} B_1 & & & \\ & B_2 & & \\ & & \boxed{B_3^{3k}} & \\ & & & B_4 \end{pmatrix}}_{\text{(d) Order probe}} \quad (6)$$

(b) Geometric loss $\rightarrow$ rank probe. Replace the $k \times k$ block B_i with a $(k+1) \times (k+1)$ block B'_i , expanding it by one row and column initialized on the identity. The loss difference $\mathcal{L}(k) - \mathcal{L}(k+1)$ measures whether the data requires an additional geometric dimension at that layer. Between integer ranks, the representation is necessarily lossy—projecting a $(k+\frac{1}{2})$ -dimensional structure onto a k -dimensional subspace. Sweeping rank gives the loss as a function of local dimensionality, directly measuring the data’s intrinsic rank at each layer. The rank at which the loss plateau begins is the layer’s geometric capacity requirement.

(c) Topological loss $\rightarrow$ topology probe. Place εI_k at off-diagonal position (i, j) in the block matrix, creating a weak connection between blocks i and j . The gradient $\partial \mathcal{L} / \partial \varepsilon$ at that position measures whether a cross-block connection would improve performance. Negative gradient: the connection carries useful signal (strengthen it). Positive gradient: the connection introduces interference (remove it, or do not add it). Computing this gradient for all $L^2 - L$ off-diagonal positions yields a *topology sensitivity map*—a complete picture of which cross-block connections the data geometry requires. Existing architectures correspond to sparse patterns in this map: ResNet activates the first superdiagonal ($\varepsilon_{i, i+2}$), DenseNet activates the full upper triangle, and U-Net activates the anti-diagonal.

(d) Geometric loss $\rightarrow$ order probe. Replace an $m=2$ block $B_i \in \text{GL}(2d)$ with an $m=3$ block $B'_i \in \text{GL}(3d)$, allowing three states to participate in a single $\exp(A)$ step. The loss difference $\mathcal{L}(2\text{-body}) - \mathcal{L}(3\text{-body})$ measures whether the data contains irreducible 3-body interactions at that layer—relationships that cannot be efficiently relayed through pairwise steps. The rank probe tests whether the embedding dimension is adequate (pointwise extrinsic); the order probe tests whether the interaction arity is adequate (mixer extrinsic). Both are aspects of geometric loss: insufficient rank forces subspace detours, and insufficient interaction order forces the network to approximate k -body couplings through depth, spending layers on relay rather than computation.

From diagnostics to architecture search. The framework suggests parameterizing each mixer with a learnable “constraint level” that interpolates between rigid (convolution-like) and flexible

(attention-like), with regularization favoring tighter constraints. During training, relationships with discoverable structure would tighten toward rigid constraints, while genuinely unstructured relationships would remain flexible. The architecture crystallizes from data.

All three probes are perturbations of the same $D \times D$ matrix, differing only in structural position: diagonal block size (rank), off-diagonal blocks (topology), and block arity (interaction order). The gradient of the loss with respect to each perturbation direction decomposes Adam’s single gradient signal into its three structural components—precisely the decomposition that standard training conflates. This replaces the combinatorial search of NAS with geometric measurement: instead of training many architectures and comparing, one architecture is probed along three orthogonal directions to determine what the data requires. The probes are cheap relative to full training—they require only local perturbation and gradient evaluation, not retraining from scratch.

7.7 Generative Models as Perpendicular Traversal

The extrinsic probes diagnose and correct mismatch for a given task. But the perpendicular reservoir—the task-irrelevant meaning identified in Section 7.3—is not merely waste to be eliminated. Generative models exploit it: they traverse the Grassmannian geometry that discriminative training left unspecified.

Diffusion models [Ho et al., 2020, Song et al., 2021] make this most explicit. The forward process progressively rotates the data subspace until it fills the full ambient space—every direction equally occupied, zero structure. The reverse process navigates back, selecting at each step which perpendicular directions to eliminate and which to retain. The score function is the gradient on the Grassmannian pointing toward the data subspace. Diffusion works because the noise schedule parameterizes a smooth Grassmannian path, and the model learns local corrections rather than one large traversal.

Variational autoencoders [Kingma & Welling, 2014] exploit mismatch as a feature. The reconstruction error is the geometric mismatch of Section 7.2. But instead of minimizing it, the VAE learns a smooth coordinate system over the Grassmannian neighborhood. The KL divergence ensures continuity; sampling from the prior explores perpendicular directions that the encoder’s task left unspecified but that correspond to meaningful data variation.

Generative adversarial networks [Goodfellow et al., 2014] introduce coupled Grassmannian optimization. Mode collapse is the generator collapsing to a low-dimensional sub-subspace—a Grassmannian position that fools the discriminator but does not span the full data neighborhood. Training instability reflects two agents moving on the same Grassmannian with opposing gradients.

The pattern is uniform: each generative family navigates perpendicular Grassmannian directions that discriminative training leaves unspecified. The perpendicular reservoir that a task-specific architecture would discard is exactly the space generative models learn to traverse. This connects architecture search to generation: the extrinsic probes measure mismatch to *reduce* it; generative models measure the same mismatch to *navigate* it.

Remark 4 (Taxonomy of Grassmannian Traversals). *The four Grassmannian positions discussed in this paper—data, meaning, latent (intermediate representations), and unstructured (noise/ambient)—yield sixteen directed traversals. Most correspond to established model families: classification (data $\rightarrow$ meaning), unconditional generation (unstructured $\rightarrow$ data), VAE encoding (data $\rightarrow$ latent), and so on. Several cells have no established model family, particularly traversals toward the unstructured position as a learned operation. A complete enumeration is deferred to future work.*

7.8 Bijectors as Boundary Compensation

Normalizing flows used as data preprocessing—learning an invertible transform to map complex distributions to simpler ones—are another instance of mismatch compensation, concentrated at the architecture’s boundary rather than distributed across layers. The bijector’s job is to undo the gap between the data’s geometry and the architecture’s prior. The framework prescribes the alternative: encode the geometric structure directly into the primitive constraints. The bijector is not eliminated but subsumed—normalizing flow coupling layers are mixers ($m = 2$), 1×1 convolutions are pointwise transforms. The difference is that current flows choose constraints for generic expressiveness, while the framework prescribes choosing them based on measured data geometry.

8 Discussion

8.1 Implications for Large Language Models

Transformer architectures impose minimal geometric constraints on the mixer primitive (full attention permits any token to attend to any other). This is not the absence of all structure—the sequential token format, positional encoding scheme, and causal mask each impose real constraints—but it is *geometric agnosticism*: the architecture assumes little about the manifold’s metric or topology, requiring the attention heads to learn from data which relationships matter and how ordering maps to meaning. Different languages arrange the same semantic content into different grammatical orderings, and the attention mechanism must discover these conventions rather than having them built in. This explains both transformers’ data hunger and their generality. It also suggests that injecting known geometric constraints—where they can be identified—could dramatically improve efficiency without sacrificing generality on the genuinely unstructured components.

8.2 The Causal Mask as Flag Manifold

The causal (lower triangular) attention mask in autoregressive LLMs has a precise geometric interpretation within the Grassmannian framework. Token i can attend to tokens $\{1, \dots, i\}$, so its output lives in $\text{span}(V_1, \dots, V_i)$ —a subspace whose dimension grows by one per token. The nested inclusion $V_1 \subset V_2 \subset \dots$ defines a **complete flag**, and the space of such nested sequences is a flag manifold—a structured path through successive Grassmannians $\text{Gr}(1, d) \rightarrow \text{Gr}(2, d) \rightarrow \dots \rightarrow \text{Gr}(d, d)$. This reveals a second geometric constraint in the transformer beyond positional encoding: the causal mask forces monotonic dimensional growth in the visible subspace.

This growth saturates at the embedding dimension d : once $i > d$, the span is already full-rank and new tokens provide refinement (more vectors to weight over) rather than expansion (new geometric directions). For a typical LLM ($d = 4096$, context = 128K), the subspace-building phase covers only $\sim 3\%$ of the context window. The two-phase structure—building then refining—predicts task-dependent optimal context lengths determined by the task’s intrinsic dimensionality.

8.3 Limitations

The completeness result applies to *linear* invertible maps. However, when the mixer is parameterized as $M = \exp(A)$ for invertibility, the matrix exponential—the canonical map from the Lie algebra $\mathfrak{gl}(md)$ to the Lie group $\text{GL}(md)$ —introduces nonlinearity through its series expansion ($I + A + A^2/2! + A^3/3! + \dots$), providing polynomial nonlinearity of all orders simultaneously. The MNIST experiments in Section 6.3 confirm that this parameterization performs comparably to architectures with explicit activation functions and that adding activation layers on top provides no additional

benefit, suggesting the Lie group structure intrinsic to the invertible primitives subsumes hand-designed nonlinearities. A full characterization of which nonlinearities $\exp(A)$ can and cannot reproduce remains future work. When m is small (e.g., $m = 2$), the mixer may be insufficient for irreducibly higher-order interactions without impractical depth—the cost of decomposing k -body interactions into m -body steps is not bounded tightly. The “architecture as prior” framing abstracts away the interaction between architecture and optimization dynamics, which may be more complex than a static constraint analysis captures.

8.4 Data Geometry in the $k \times k$ Operating Space

The framework’s analysis takes place in a $k \times k$ block of $\text{GL}(md)$, but real data rarely populates this space. An RGB image token carries $r = 5$ intrinsic dimensions (three color channels plus two position coordinates in the unified state vector); a text token is a one-hot index projected into $d = 4096$ embedding dimensions. In both cases, the input occupies an r -dimensional submanifold of the d -dimensional feature space, with $r \ll d$ —a gap confirmed empirically: learned representations have intrinsic dimension orders of magnitude below layer width [Ansuini et al., 2019], and fine-tuning a language model requires as few as 200 trainable parameters to recover 90% of full performance [Aghajanyan et al., 2021]. Sharp readers will notice that most of the paper’s geometric discussion implicitly assumes a richly populated $k \times k$ block—a condition that no standard input representation satisfies under the unified state vector.

This gap admits two boundary interpretations, each corresponding to a distinct regime of the primitives.

The Gram boundary: all geometry at $\exp(0)$. The outer product $xx^\top$ populates the $k \times k$ block with every pairwise geometric relationship among tokens—inner products, angles, relative magnitudes—at *zero learned cost*. This is the mixer primitive evaluated at $A = 0$ (i.e., $M = \exp(0) = I$): no learned rotation is required because the Gram matrix exhaustively encodes the data’s relational structure. Permutation is implicitly total (every token pairs with every token through the inner product) and the transformation from x to $x^\top$ is the maximal pointwise repackaging of input features. Attention computes exactly this: $QK^\top$ is a learned variant of $xx^\top$. The Gram matrix is rank- r , so the $k \times k$ block is populated but rank-deficient—all relational geometry is present, but confined to an r -dimensional submanifold. That attention matrices are empirically low-rank is well established [Wang et al., 2020]; pure self-attention without skip connections loses rank doubly exponentially with depth [Dong et al., 2021], and rank collapse in width worsens as context length grows [Barbero et al., 2025]. In the present framework, these are not pathologies but consequences of operating near the Gram boundary: the $k \times k$ block *should* be rank- r when the data is rank- r , and the network’s challenge is deciding when to stay there and when to leave.

The normal-bundle boundary: empty space as workspace. At the opposite extreme, the $(d - r)$ -dimensional complement of the data’s column space—the normal bundle of the input submanifold—remains unpopulated. No input signal provides gradient information in these directions. In the language of the primitives, this regime assumes minimal interaction: the mixer has not yet rotated information into the empty directions, and the pointwise transform has not expanded the data’s rank. The normal bundle is geometrically inert until learning actively populates it.

Primitive roles across the spectrum. The two boundaries assign distinct roles to each primitive. The *pointwise transform* is the rank operator: it mediates between the data’s intrinsic r

dimensions and the full d , embedding low-rank inputs into the operating space (expanding rank) or compressing representations back down (reducing rank). The *permutation* selects which tokens enter the $k \times k$ block, determining the support of $xx^\top$. The *mixer* operates on the block itself: in intrinsic mode it rearranges relational structure within the rank- r Gram manifold; in extrinsic mode it lifts to md dimensions and can traverse the normal bundle, creating correlations in directions the input did not populate.

This decomposition predicts that tasks whose relevant structure is fully captured by pairwise token relationships—classification, retrieval, semantic similarity—can operate near the Gram boundary, relying on intrinsic mixing to refine the relational geometry that $xx^\top$ provides for free. Generative tasks, which must produce outputs in directions absent from the input, require large departures from $\exp(0)$: extrinsic mixing to traverse the normal bundle and rank-expanding pointwise transforms to make those directions available. The “difficulty” of a task, in this view, maps onto its distance from the identity in the primitive parameter space. This is consistent with the observation that transformer representations pass through a high-dimensional abstraction phase in intermediate layers before compressing toward the output [Valeriani et al., 2024]—the network departs from the Gram boundary (expanding into the normal bundle) before returning to a low-rank meaning subspace. It also connects to the lottery ticket hypothesis [Frankle & Carlin, 2019]: the sparse subnetworks that suffice for a given task are precisely those whose primitives need only small departures from the identity, while the pruned parameters correspond to normal-bundle directions the task never requires. More broadly, the geometric transformation of object manifolds across layers—from non-separable to linearly separable [Cohen et al., 2020]—can be understood as the progressive reshaping of the rank- r Gram structure by learned mixer and pointwise operations.

8.5 Restoring the Position/Feature Distinction

The dissolution of position and features (Section 4) is the framework’s most general setting, but practical architectures often benefit from maintaining the separation. This is not a limitation of the three-primitive framework—it is a constraint choice within it.

Convolutions are the clearest example. The CNN deliberately keeps position dimensions separate from feature dimensions because translation equivariance requires the mixer to depend only on positional differences: $M_{ij} = M(s_i^{(\text{pos})} - s_j^{(\text{pos})})$. Enforcing this constraint is what makes convolutions computationally efficient—local, weight-shared, and parallelizable. The position/feature split is not geometrically necessary (attention over the same data dissolves it) but architecturally advantageous when the data’s geometry is known to be translation-invariant. The split is a speed choice: it encodes a known symmetry directly into the primitive constraints rather than requiring the network to rediscover it.

The same logic applies more formally in gauge-equivariant settings [Cohen et al., 2019, Weiler et al., 2021]. The fiber bundle formalism requires a base space (position) distinct from the fiber (features) to state gauge constraints precisely, introducing dedicated geometric operations—logarithmic maps ($\log_{p_i}(p_j) \in T_{p_i}\mathcal{M}$), metric evaluations ($g_i(u, w) = u^T G_i w$)—because position is treated as a separate geometric object. In the unified state vector, these map directly to primitive constraints: displacement computation becomes a constraint on how the mixer depends on position-dimensions, metric evaluation becomes a constraint on the mixer’s inner product structure ($q^T k$ is Euclidean; $q^T G k$ is non-Euclidean), and gauge transforms are pointwise transforms on the full state. Problems whose geometry is well-described by a known Riemannian manifold with a gauge group—mesh processing on known surfaces, molecular dynamics with known symmetries—benefit from restoring the distinction and imposing these constraints explicitly.

In the language of Section 4.1, both cases amount to fixing a specific constraint pattern that

treats position and feature dimensions asymmetrically. The framework accommodates this: architectures that maintain the position/feature separation are points in the constraint space where that separation is enforced as a structural prior. The unified formulation is the more general setting; the separated formulation is the appropriate specialization when the manifold structure is known a priori and the computational savings of exploiting it are worth the reduced generality.

9 Conclusion

We have shown that all major neural network architectures—convolutional, attentional, graph, spherical, and hyperbolic—are specific constraint configurations of three known reversible primitives: permutation, pointwise transform, and invertible mixer. The primitives and their intrinsic completeness are not new; they follow from classical results (LU decomposition, butterfly factorization). What is new is the universal characterization: every architecture maps to a point in a single continuous constraint space, and the 3×2 grid of intrinsic and extrinsic modes cleanly decomposes every major architecture component—including multi-head attention—without additional primitives. The extrinsic modes provide a Grassmannian interpretation: a network’s computation is a path on the Grassmannian from the data’s initial subspace to the task’s meaning space, and the architecture’s constraints determine which paths are accessible. A correct architecture is one whose constraints make a short, direct path available.

The central implication is that architecture development itself—historically a process of human intuition, empirical benchmarking, and convention—is optimization over a well-defined geometric space. The constraint parameters that distinguish convolution from attention from graph message passing are continuous and differentiable, not discrete design choices. This means the question “which architecture should I use?” can in principle be answered by measuring the data’s geometry and selecting constraints that minimize the Grassmannian path length to the meaning subspace.

The Grassmannian perspective also explains why overparameterization helps (larger ambient space provides geometric headroom for subspace correction), why early layers learn general features (they perform larger subspace rotations to correct embedding errors), and why matched architectures are data-efficient (shorter Grassmannian paths require fewer layers to traverse). The framework dissolves the artificial separation between position and features, reframes latent space as a measurement of geometric mismatch between architecture and data, and unifies discriminative and generative models as opposite traversal directions on the same manifold. The prior is good when the constrained family contains short paths from data space to meaning space.

A Supplementary MNIST Results

A.1 Pixel Ordering Invariance (Test 5)

A standard concern with non-convolutional architectures is dependence on input ordering. We compare row-major pixel ordering (the default MNIST flattening) against a fixed random permutation of all 784 pixels.

The gap is less than 1%, within noise. The model reads geometry from the normalized (x, y) coordinates encoded in each state vector $[x, y, \text{value}]$, not from index adjacency in the routing topology. This validates the unified encoding design: position is carried as data, not as structural assumption.

Table 11: Pixel ordering comparison. Mean $\pm$ std over 3 seeds.

Ordering	Accuracy	Std
Row-major	79.17%	1.47%
Random	79.78%	1.50%
Gap	-0.62%	—

A.2 Receptive Field Analysis

Gradient-based effective receptive field comparison (center pixel, averaged over 20 samples) confirms that the three architectures build fundamentally different spatial connectivity:

- **Primitives (depth 5):** 82.1% accuracy, 1345 params. Sparse long-range connections via routing topology. Exactly 5 bright spots visible in the gradient map (one per routing+mixing step), reflecting the 5 routing patterns: butterfly, bit-reversal, stride, cyclic shift, butterfly.
- **CNN 1-layer:** 47.4% accuracy, 1330 params. Nearly invisible receptive field—a single convolution layer sees only a tiny local patch, insufficient for MNIST digit recognition.
- **CNN 2-layer:** 86.8% accuracy, 1330 params. Smooth local patch around center pixel, bounded by effective kernel size. Well-matched to MNIST’s local spatial structure.

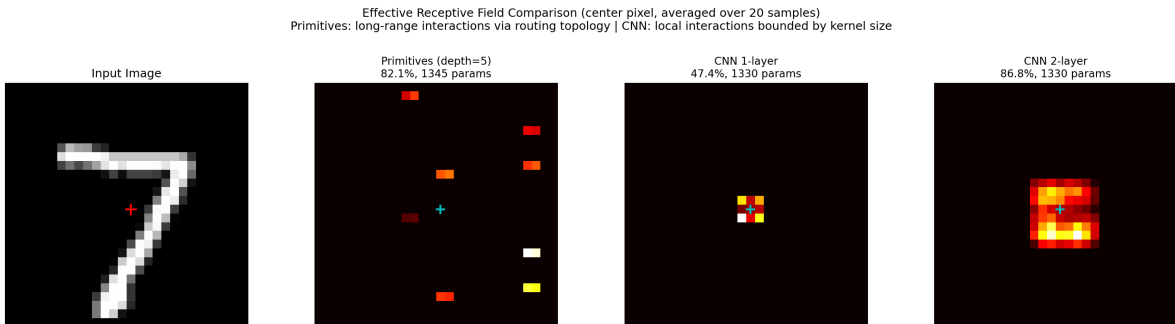


Figure 13: Gradient-based effective receptive fields for three architectures at matched parameter count (~ 1330 params). Left: Primitives (depth 5) show sparse long-range connections via routing topology. Center: 1-layer CNN has a nearly invisible receptive field. Right: 2-layer CNN shows a smooth local patch.

A 7×7 receptive field grid (4px spacing, per-cell normalization) reveals that the routing topology creates a fixed pattern of long-range connections regardless of target pixel position. Edge pixels show gradient magnitudes up to 2.3×10^6 (vs. $\sim 6 \times 10^3$ at center) due to numerical instability through deep $\exp(A)$ chains in regions with minimal training signal—a known challenge for deep invertible architectures that future work on gradient clipping strategies may address.

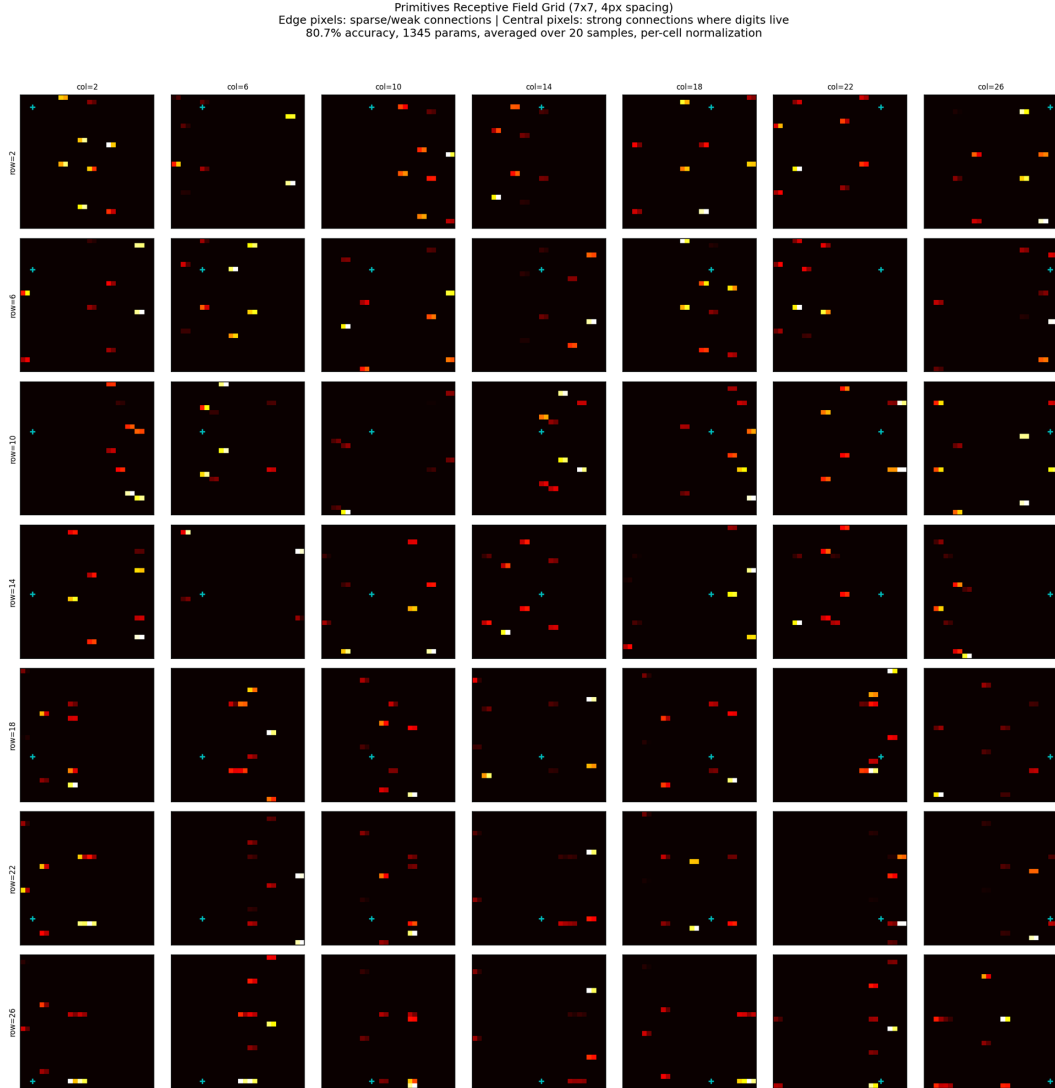


Figure 14: Receptive field grid (7×7 , 4px spacing, per-cell normalization). The routing topology creates a fixed pattern of long-range connections regardless of target pixel position.

The gradient distribution aggregated over 16 target pixels shows: primitives produce a flat, sparse distribution (few pixels with moderate gradients, most near zero); the 1-layer CNN concentrates nearly all gradient mass at zero (tiny receptive field); the 2-layer CNN produces a smooth tail (broader effective receptive field with many small-to-moderate gradient contributions).

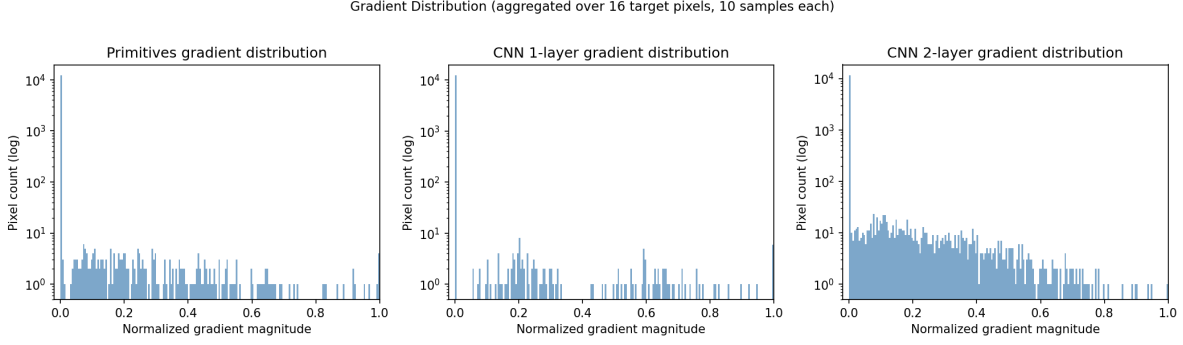


Figure 15: Gradient magnitude distributions aggregated over 16 target pixels. Primitives produce a flat, sparse distribution; the 1-layer CNN concentrates gradient mass at zero; the 2-layer CNN produces a smooth tail reflecting its broader effective receptive field.

B Supplementary Figures

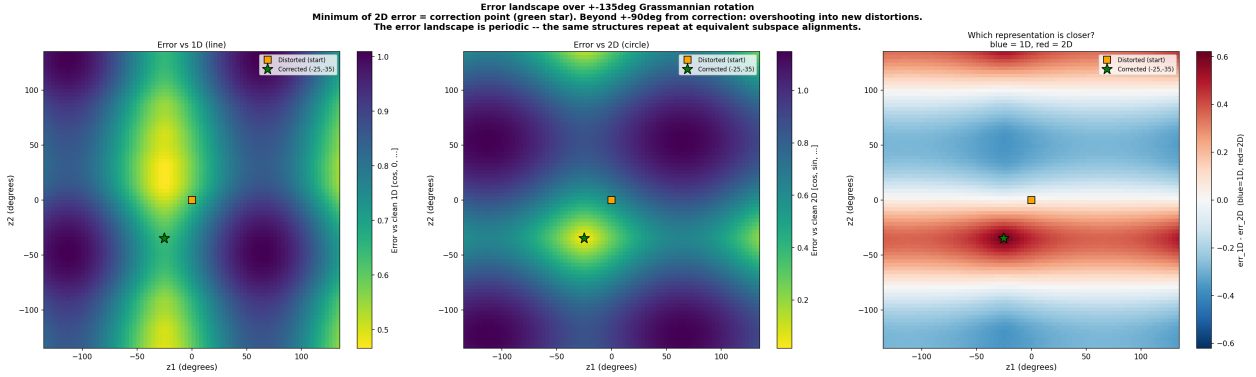


Figure 16: High-resolution Grassmannian error landscape over $\pm 135^\circ$. Left: error relative to 1D representation (minimum is a band—many subspaces contain e_x). Center: error relative to 2D representation (minimum sharply localized at correction point). Right: the difference reveals which Grassmannian regions are better described as 1D versus 2D. The periodic structure reflects the compactness of $\text{Gr}(2, 4)$.

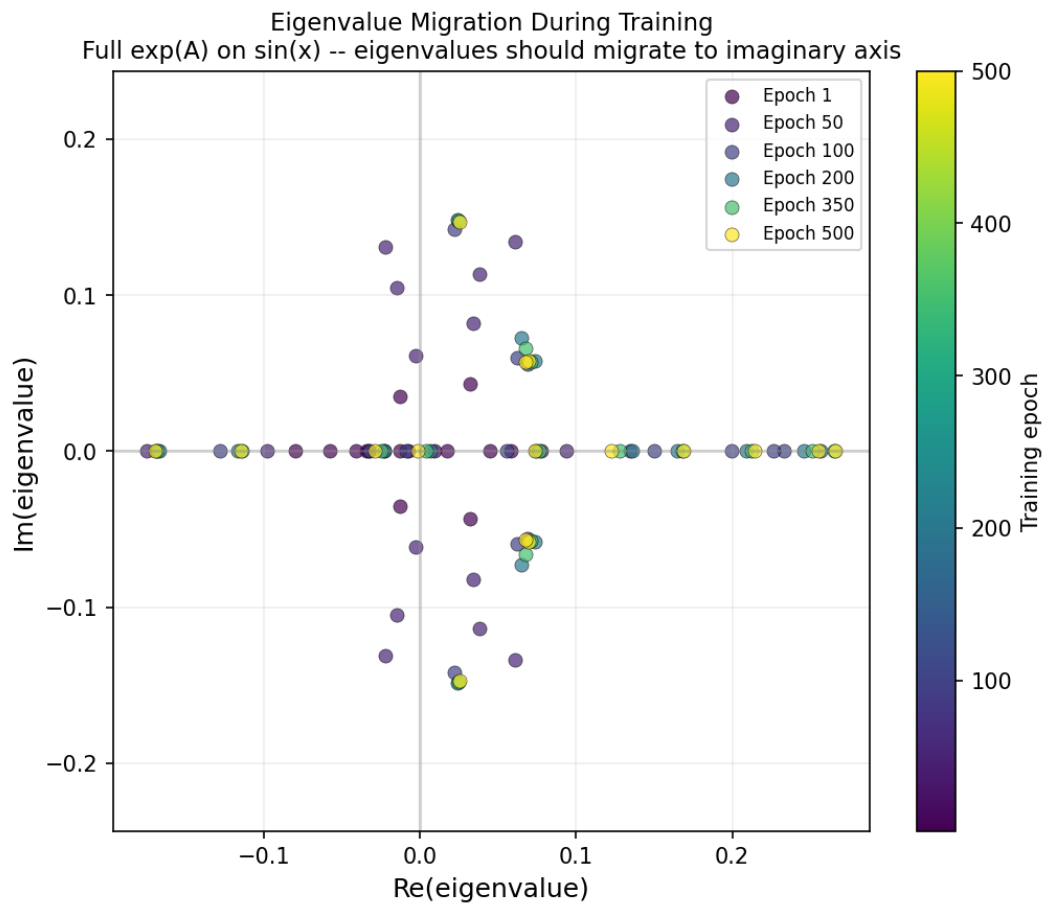


Figure 17: Eigenvalue migration during training on $\sin(x)$ (full $\exp(A)$, 8 layers). Eigenvalues begin near the real axis (epoch 1, dark) and migrate toward the imaginary axis (epoch 500, bright), developing the conjugate pairs needed for sinusoidal representation. The network discovers rotational structure autonomously.

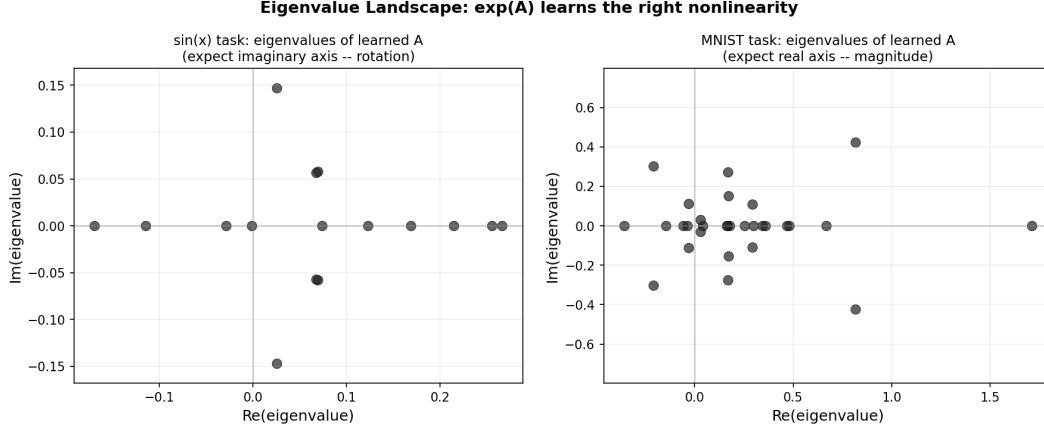


Figure 18: Eigenvalue landscape comparison between tasks. Left: $\sin(x)$ task, where eigenvalues cluster near the imaginary axis (rotation-dominated, as expected for periodic structure). Right: MNIST task, where eigenvalues spread along the real axis (magnitude-dominated, reflecting classification’s reliance on feature scaling). The task geometry determines which mode of $\exp(A)$ the network exploits.

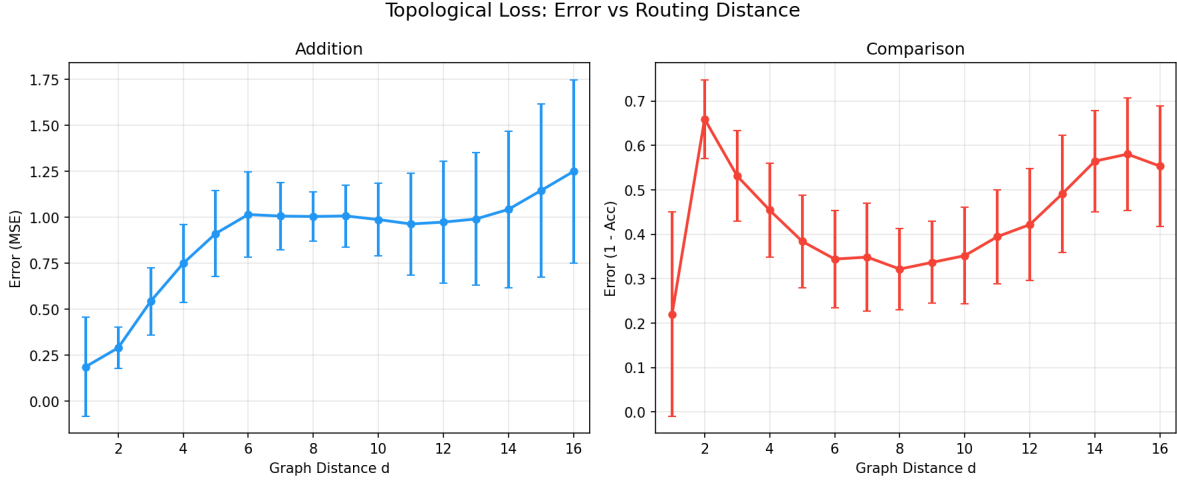


Figure 19: Topological distance curve. Addition error (MSE) and comparison error ($1 - \text{accuracy}$) vs. graph distance d between the two operand tokens. Addition degrades monotonically until saturating at chance; comparison remains above chance at all distances due to single-operand discriminability. 5 seeds, 10 random position pairs per seed.

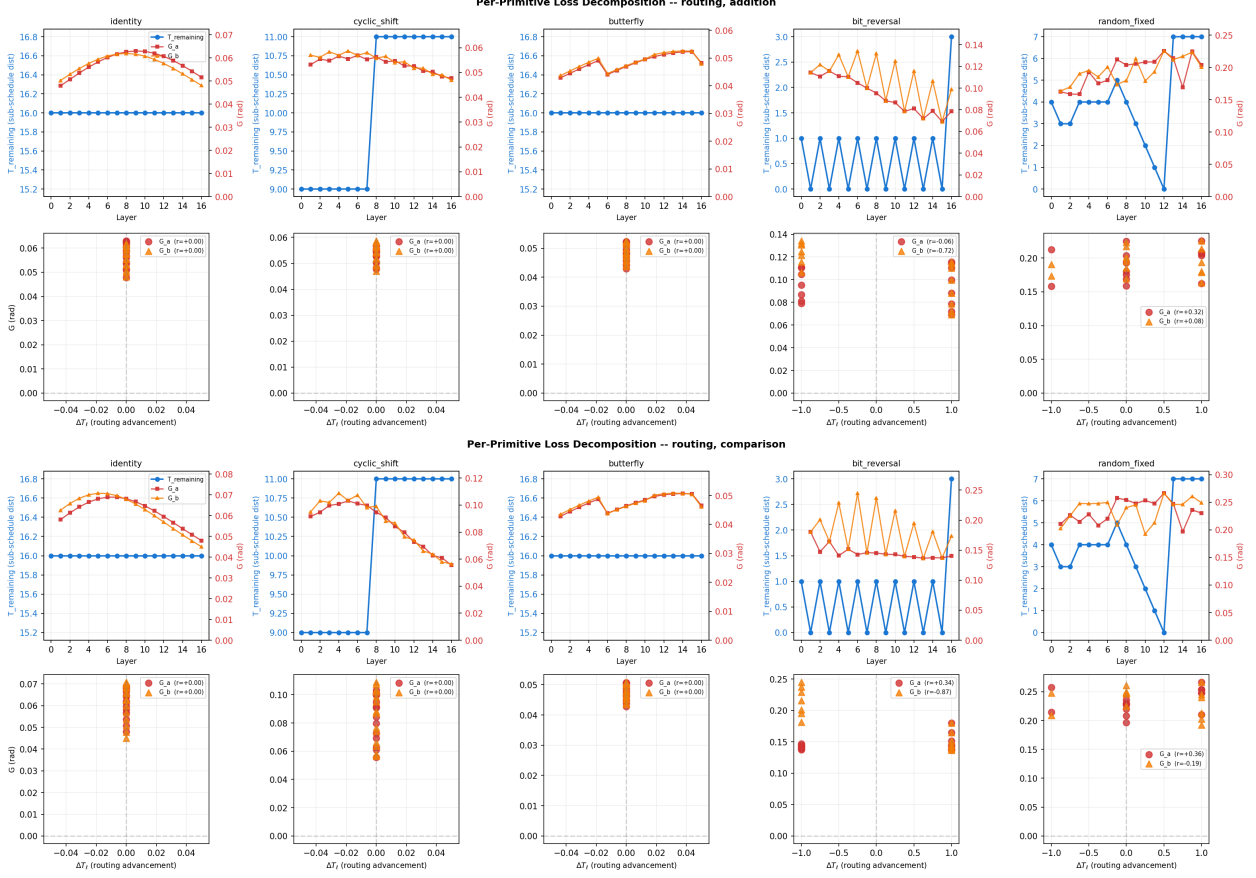


Figure 20: Per-primitive decomposition across routing topologies (identity, cyclic shift, butterfly, bit-reversal, random fixed) on the fixed pair (0, 16). Bit-reversal on comparison shows $r_b = -0.87$: the mixer rotates less when routing advances and more when routing idles, reflecting surplus depth after the $d=2$ relay completes in 16 layers. Addition (top) and comparison (bottom).

References

- M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.
- T. S. Cohen, M. Weiler, B. Kicanaoglu, and M. Welling. Gauge equivariant convolutional networks and the icosahedral CNN. In *ICML*, 2019.
- N. Dehmamy, R. Walters, Y. Liu, D. Chen, and R. Yu. Automatic symmetry discovery with Lie algebra convolutional network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- M. Weiler, P. Forré, E. Verlinde, and M. Welling. Equivariant and coordinate independent convolutional networks: A gauge field theory of neural networks. 2021.
- T. Dao, A. Gu, M. Eichhorn, A. Rudra, and C. Ré. Learning fast algorithms for linear transforms using butterfly factorizations. In *ICML*, 2019.
- T. Dao, N. Sohoni, A. Gu, M. Eichhorn, A. Rudra, and C. Ré. Kaleidoscope: An efficient, learnable representation for all structured linear maps. In *ICLR*, 2020.

- T. Dao, B. Chen, N. Sohoni, A. Desai, M. Poli, J. Grogan, A. Liu, A. Rao, A. Rudra, and C. Ré. Monarch: Expressive structured matrices for efficient and accurate training. In *ICML*, 2022.
- L. Dinh, D. Krueger, and Y. Bengio. NICE: Non-linear independent components estimation. *arXiv preprint arXiv:1410.8516*, 2014.
- L. Dinh, J. Sohl-Dickstein, and S. Bengio. Density estimation using Real-NVP. *arXiv preprint arXiv:1605.08803*, 2016.
- R. T. Q. Chen and Y. Lipman. Flow matching on general geometries. In *ICLR*, 2024.
- Authors. Geometric inductive biases of deep networks: The role of data and architecture. *arXiv preprint arXiv:2410.12025*, 2024.
- J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In *ICLR*, 2021.
- D. P. Kingma and M. Welling. Auto-encoding variational Bayes. In *ICLR*, 2014.
- I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *NeurIPS*, 2014.
- Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013.
- N. Tishby, F. C. Pereira, and W. Bialek. The information bottleneck method. *arXiv preprint physics/0004057*, 2000.
- I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner. β -VAE: Learning basic visual concepts with a constrained variational framework. In *ICLR*, 2017.
- C. Meng, L. Yang, R. Zhuo, and S. Ermon. ButterflyFlow: Building invertible layers with butterfly matrices. In *ICML*, 2022.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- Z. Huang, J. Wu, and L. Van Gool. Building deep networks on Grassmann manifolds. In *AAAI*, 2017.
- Z. Chong. Attention is not what you need: Grassmann flows as an attention-free alternative for sequence modeling. *arXiv preprint arXiv:2512.19428*, 2025.
- H. Liu, K. Simonyan, and Y. Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations (ICLR)*, 2019.
- D. Hendrycks and K. Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.
- T. Bendokat, R. Zimmermann, and P.-A. Absil. A Grassmann manifold handbook: Basic geometry and computational aspects. *Advances in Computational Mathematics*, 50(6), 2024.

- K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2261–2269, 2017.
- O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 234–241, 2015.
- A. Ansuini, A. Laio, J. H. Macke, and D. Zoccolan. Intrinsic dimension of data representations in deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- A. Aghajanyan, S. Gupta, and L. Zettlemoyer. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 7319–7328, 2021.
- L. Valeriani, D. Doimo, F. Cuturello, A. Laio, A. Ansuini, and A. Ceni. Emergence of a high-dimensional abstraction phase in language transformers. *arXiv preprint arXiv:2405.15471*, 2024.
- Y. Dong, J.-B. Cordonnier, and A. Loukas. Attention is not all you need: Pure attention loses rank doubly exponentially with depth. In *International Conference on Machine Learning (ICML)*, pp. 2793–2803, 2021.
- F. Barbero, A. Banino, S. Kapturowski, D. Kumaran, and P. Veličković. Mind the gap: A spectral analysis of rank collapse and signal propagation in attention layers. In *International Conference on Learning Representations (ICLR)*, 2025.
- S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.
- J. Frankle and M. Carlin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations (ICLR)*, 2019.
- U. Cohen, S. Y. Chung, D. D. Lee, and H. Sompolinsky. Separability and geometry of object manifolds in deep neural networks. *Nature Communications*, 11(1):746, 2020.